Relatório - Projeto 2 - ICC2

André Campanholo Paschoalini
Nº**USP - 16862892**

Pedro Hamamoto da Palma
Nº**USP - 16818280**

Data: 20/11/2025

Prof. Marcelo Garcia Manzato

1. Introdução

O projeto proposto consiste em implementar e testar, com vetores de tamanhos e organizações variadas, os seguintes algoritmos de ordenação: **BubbleSort**, **SelectionSort**, **InsertionSort**, **ShellSort**, **QuickSort** (utilizando mediana de 3 para identificar o pivô), **HeapSort**, **MergeSort**, **Contagem dos Menores** e **RadixSort.**

Para cada execução de um algoritmo de ordenação, devem ser contabilizados (utilizando contadores):

- O número de comparações de chaves;
- O número de movimentações de registros;
- O tempo de execução do algoritmo.

Os vetores a serem ordenados podem ter os seguintes tamanhos: **100**, **1.000**, **10.000** ou **100.000** elementos. Tais vetores podem, também, estar **ordenados**, **inversamente ordenados**, ou com **registros aleatórios**. Cada método de ordenação foi testado com todas as combinações possíveis de características de vetor.

Nesse relatório, serão discutidas decisões de implementação, resultados empíricos e a comparação desses resultados com os resultados esperados segundo a análise algorítmica.

2. Algoritmos utilizados

2.1 BubbleSort

O **Bubble Sort** é um algoritmo de ordenação simples que compara pares **adjacentes** e os troca quando estão fora de ordem. A cada passada pelo vetor, o maior elemento ainda não posicionado “sobe” para o final, como uma bolha. O processo se repete até que todo o vetor esteja ordenado.

Na versão clássica, sempre são feitas todas as passadas; na versão otimizada(implementada nesse projeto), o algoritmo pode parar mais cedo se nenhuma troca ocorrer em uma iteração.

Comparações

No algoritmo clássico, o número total de comparações é sempre $\frac{n(n-1)}{2}$independentemente da ordem inicial dos dados. Já no otimizado, esse número de comparações depende do estado do vetor.

Trocas

- **Melhor caso (vetor já ordenado):** 0 trocas.
- **Pior caso (vetor invertido):** ($\frac{n(n-1)}{2}$) trocas.
- **Caso médio:** aproximadamente ($\frac{n(n-1)}{4}$) trocas.

Complexidade de tempo

- **Otimizado:** melhor caso → $O(n)$; médio e pior → $O(n^2)$

Complexidade de espaço

O algoritmo é **in-place**, utilizando apenas memória auxiliar constante $O(1)$

O Bubble Sort é estável e fácil de implementar, porém pouco eficiente para vetores grandes devido ao seu custo quadrático. Na prática, ele é utilizado apenas de maneira didática.

2.2 SelectionSort

O **Selection Sort** é um algoritmo de ordenação que, a cada passada pelo vetor, **seleciona o menor elemento** da parte ainda não ordenada e o coloca na posição correta realizando **uma única troca por iteração externa**. Assim, após a primeira passada o menor valor estará em `v[0]`, após a segunda o segundo menor em `v[1]`, e assim por diante.

Comparações

Em todas as situações (melhor, médio e pior caso), o algoritmo sempre percorre o restante do vetor para encontrar o menor elemento. O número total de comparações é $\frac{n(n-1)}{2}$.

Trocas

O Selection Sort troca apenas uma vez por iteração externa, independentemente da ordem inicial:

- **Número total de trocas:** $n-1$, desde que troque mesmo quando o menor já está na posição correta (implementação comum).
- Em versões que evitam trocas desnecessárias, o número pode ser menor no melhor caso, mas permanece $O(n)$

Complexidade de tempo

Como o número de comparações não depende da ordem inicial:

- **Melhor caso:** $O(n^2)$
- **Caso médio:** $O(n^2)$
- **Pior caso:** $O(n^2)$

Complexidade de espaço

O algoritmo é **in-place**, usando apenas memória auxiliar constante $O(1)$

O Selection Sort é simples, tem número reduzido de trocas e comportamento determinístico, mas é pouco eficiente para vetores grandes devido ao custo quadrático de comparações.

2.3 InsertionSort

O **Insertion Sort** constrói a ordenação de forma incremental: percorre o vetor da esquerda para a direita e, para cada elemento, "insere" esse valor na posição correta da parte já ordenada, deslocando elementos maiores para a direita.

Na implementação fornecida, a cada iteração do laço externo o elemento `v[i]` é armazenado em `elem`, e o laço interno percorre a sublista ordenada à esquerda, realizando comparações e deslocamentos até encontrar a posição correta onde `elem` será inserido.

Comparações

A quantidade de comparações depende fortemente da ordem inicial:

- **Melhor caso (vetor já ordenado):** $O(n)$ comparações

  Cada elemento é comparado uma única vez e o laço interno interrompe imediatamente.

- **Pior caso (vetor em ordem inversa):** $O(n^2)$

  Para cada `i`, são feitas `i` comparações.
- **Caso médio:** $O(n^2)$ comparações.

  Aproximadamente metade da sublista é percorrida.

Trocas

- **Melhor caso:** n trocas(uma por iteração).
- **Pior caso (ordem inversa):** Trocas = $O(n^2)$

  Na i-ésima iteração são feitos `i` deslocamentos mais a gravação final:
- **Caso médio:** $O(n^2)$

Complexidade de tempo

- **Melhor caso:** $O(n)$
- **Caso médio:** $O(n^2)$
- **Pior caso:** $O(n^2)$

Complexidade de espaço

O algoritmo é **in-place**, usando apenas variáveis auxiliares. Assim, complexidade de espaço é $O(1)$

O Insertion Sort é eficiente para vetores quase ordenados e estável, com ótimo desempenho no melhor caso, mas apresenta custo quadrático em cenários desfavoráveis.

2.4 ShellSort

O **ShellSort** é uma extensão do Insertion Sort que utiliza gaps (incrementos) decrescentes para realizar inserções "por saltos". Primeiro são escolhidos incrementos grandes que são aplicados do maior para o menor. Para cada gap, o vetor é percorrido e cada elemento é inserido na posição correta dentro do subvetor formado pelos índices separados por h.

As comparações aumentam a cada teste entre o elemento em inserção e o elemento h posições atrás; as trocas contam cada deslocamento para a direita e a escrita final do elemento.

Comparações

- **Melhor caso:** $O(n * log(N))$ — cada inserção interrompe rapidamente.
- **Caso médio / pior:** entre $O(n^{3/2})$ e $O(n^2)$, dependendo da distribuição inicial.

Trocas

- **Melhor caso:** $O(n * log(n))$
- **Caso médio / pior:** mesma ordem das comparações, entre $O(n^{3/2})$ e $O(n^2)$.

Complexidade de tempo

- **Melhor:** $O(n * log(n))$
- **Médio / pior:** cerca de $O(n^{3/2})$ com a sequência $3h + 1$.

Espaço

O algoritmo é **in-place**, exigindo apenas variáveis auxiliares, sendo assim $O(1)$

O ShellSort, assim, reduz drasticamente o custo do Insertion Sort em vetores grandes, mantendo implementação simples e bom desempenho prático.

2.5 QuickSort

O **QuickSort** é um algoritmo de ordenação recursivo que escolhe um pivô, particiona o vetor em dois subvetores (elementos menores à esquerda e maiores à direita) e aplica o mesmo processo recursivamente até que todos os segmentos tenham tamanho 0 ou 1.

Aqui é adotada a estratégia de **mediana de três** para escolher o pivô: toma-se o primeiro elemento, o elemento central e o último, ordenam-se esses três valores, e a mediana é usada como pivô. Isso tende a evitar divisões muito desbalanceadas em vetores já ordenados ou quase ordenados.

Comparações

- **Melhor caso (partições bem equilibradas):** $O(n * log(n))$
- **Caso médio:** $O(n * log(n))$
- **Pior caso:** ainda $O(n^2)$, apesar da mediana de três tornar essa situação rara na prática.

Trocas

- **Melhor / caso médio:** número de trocas proporcional ao de comparações, em torno de $O(n * log(n))$
- **Pior caso:** $O(n^2)$

Complexidade de tempo

- **Melhor:** $O(n * log(n))$
- **Médio:** $O(n * log(n))$
- **Pior:** $O(n^2)$

Complexidade de espaço

QuickSort é **in-place**, usando apenas a pilha de recursão:

- profundidade típica: $O(log(n))$
- pior caso: $O(n)$

Em resumo, o QuickSort com pivô escolhido por mediana de três combina baixo uso de memória com excelente desempenho prático, mantendo tempo O(nlogn)O(n \log n)O(nlogn) na maioria dos casos.

2.6 HeapSort

O **HeapSort** é um algoritmo de ordenação baseado na estrutura de dados heap binário máximo. Ele funciona em duas etapas principais:

1. **Construção do heap**: transforma o vetor em um heap máximo, onde cada pai é maior que seus filhos.
2. **Processo de ordenação**: troca o maior elemento (na raiz do heap) com a última posição ainda não ordenada e reduz o tamanho do heap. Em seguida, restaura a propriedade de heap (**heapify**) e repete até o vetor inteiro estar ordenado.

Esse método garante que, a cada iteração, o maior elemento restante é colocado corretamente no final.

Comparações

A maior parte das comparações ocorre durante as operações de **heapify**, enquanto o elemento desce pela árvore:

- **Construção do heap:** $O(n)$ comparações.
- **Fase de ordenação:** cada remoção da raiz custa $O(log(n))$ comparações.
- Total: $O(n * log(n))$

Trocas

Trocas ocorrem quando:

- o maior elemento é movido para o fim do vetor;
- durante o heapify, quando o nó é trocado com seu maior filho.

No total:

- **Total típico:** $O(n * log(n))$ trocas.
- **Nenhum caso especial reduz drasticamente o número de trocas**, pois sempre há, no mínimo, a troca da raiz com o fim em cada iteração.

Complexidade de tempo

- **Melhor:** $O(n * log(n))$
- **Médio:** $O(n * log(n))$
- **Pior:** $O(n * log(n))$

  HeapSort mantém a mesma complexidade para todos os casos, já que o heapify sempre tem custo proporcional à altura da árvore.

Complexidade de espaço

O algoritmo é **in-place**, precisando apenas de algumas variáveis auxiliares, tendo complexidade $O(1)$

O HeapSort garante desempenho $O(n * log(n))$ em qualquer cenário, possui bom número de comparações e trocas e não sofre com piores casos tão ruins como o QuickSort. Apesar disso, é geralmente um pouco mais lento que algoritmos com a mesma complexidade, como o QuickSort.

2.7 MergeSort

O **MergeSort** é um algoritmo de ordenação recursivo do tipo dividir e conquistar: o vetor é dividido ao meio repetidas vezes até obter subvetores de tamanho 1, e depois essas partes são intercaladas (merge) em ordem crescente até reconstruir o vetor completo.

Na implementação considerada, na etapa de *merge* cada metade é copiada para vetores auxiliares (esquerda e direita), e depois esses dois vetores são intercalados de volta para o vetor original, sempre escolhendo o menor elemento disponível.

- Cada comparação entre elementos das metades incrementa `num_comps`.
- Cada escrita de um elemento de volta no vetor original incrementa `num_swaps`.

Comparações

Durante a intercalação de duas partes com tamanhos $n1$ e $n2$, o número de comparações fica entre $min(n1, n2)$ e $n1 + n2 - 1$.

Somando todos os níveis de recursão:

- **Melhor caso:** $O(n * log(n))$
- **Caso médio:** $O(n * log(n))$

- **Pior caso:** $O(n * log(n))$

A ordem de complexidade é a mesma em todos os casos, pois a divisão em metades independe da ordem dos dados.

Trocas (cópias de volta ao vetor)

No *merge*, cada elemento é escrito **exatamente uma vez** de volta no vetor original, e cada escrita é contada em `num_swaps`. Em cada nível da recursão, são escritas n posições ao todo, e há cerca de $log(n)$ níveis:

- **Melhor, médio e pior caso:**$O(n * log(n))$

Complexidade de tempo

- **Melhor:** $O(n * log(n))$
- **Médio:** $O(n * log(n))$
- **Pior:** $O(n * log(n))$

Essa estabilidade de custo é uma das principais características do MergeSort.

Complexidade de espaço

A intercalação usa vetores auxiliares para guardar as metades (`esq` e `dir`), somando, em cada chamada, um espaço proporcional ao tamanho do trecho sendo mesclado. A quantidade máxima de memória extra usada simultaneamente é proporcional a n, ou seja, $O(n)$.

Além disso, há o custo da pilha de recursão, com profundidade $O(log(n))$

Em resumo, o MergeSort oferece tempo $O(n * log(n))$ em qualquer caso, à custa de memória adicional $O(n)$, e, com a condição `esq[i] <= dir[j]`, é um algoritmo **estável**.

2.8 Contagem de Menores (RankSort)

O **RankSort** (algoritmo de contagem de menores) ordena o vetor calculando, para cada elemento, quantos valores são menores ou iguais a ele e usando essa contagem como uma espécie de "posição ordenada".

O funcionamento é, em etapas:

1. Inicialização: cria dois vetores auxiliares `aux` (para as contagens) e `sorted` (vetor ordenado), ambos de tamanho `n`.
2. Contagem de menores/iguais: para cada posição `i`, percorre todo o vetor (`j = 0..n-1`) e conta em `aux[i]` quantos elementos `v[j] <= v[i]` com `j != i`. Esse valor é o "rank" do elemento.
3. Construção do vetor ordenado: para cada `i`, tenta colocar `v[i]` em `sorted[aux[i]]`.
   - Se essa posição estiver vazia, escreve ali.
   - Se já estiver ocupada (caso de empates), anda para índices anteriores até achar uma posição vazia e grava o valor.

     Cada gravação em `sorted` é contabilizada em `num_swaps`.
4. Cópia final: o vetor `sorted` é copiado de volta para `v` (sem incrementar o contador de trocas).

Comparações

O contador `num_comps` é incrementado **uma vez em cada iteração do laço duplo** (para cada par `(i, j)`), antes do teste `if (v[j] <= v[i] && i != j)`.

Há dois laços indo de `0` a `n-1.` Portanto, em melhor, médio e pior caso, o número de comparações é $O(n^2)$.

Trocas (escritas contadas)

No código, `num_swaps` é incrementado **somente** quando um elemento é escrito em `sorted`:

Cada elemento `v[i]` é gravado exatamente **uma vez** em `sorted` (seja diretamente em `sorted[aux[i]]`, seja depois de andar para a esquerda).

Assim, o número de "trocas" contadas é $O(n)$ independente da ordem inicial dos dados.

Complexidade de tempo

- O laço duplo da fase de contagem domina o custo: $O(n^2)$.
- Na fase de construção de `sorted`, em casos com muitos empates pode haver um `while` interno percorrendo posições para trás, mas isso não muda a ordem assintótica: permanece $O(n^2)$. Assim, para todos os casos, o algoritmo mantém essa complexidade temporal

Complexidade de espaço

O algoritmo usa dois vetores auxiliares `aux` e `sorted`, ambos de tamanho `n`. Assim, a complexidade temporal é $O(n)$.

Em resumo, o RankSort baseado em contagem de menores é fácil de entender e implementar, mas tem **tempo quadrático** $O(n^2)$ e requer **memória adicional** $O(n)$, sendo menos eficiente que as outras alternativas e sendo usado, na prática, somente para fins didáticos.

2.9 RadixSort

O **RadixSort** ordena os números "dígito a dígito", começando pela unidade, depois dezena, centena etc.

Para cada posição decimal, ele usa um **Counting Sort estável** aplicado apenas àquele dígito:

1. Encontra o maior elemento do vetor para saber quantos dígitos serão necessários.
2. Para cada potência de 10 (`exp = 1, 10, 100, ...`), chama o `countingSortForRadix`, que:
   - conta quantas vezes cada dígito (0–9) aparece;
   - acumula essas contagens;
     - monta um vetor `output` estável, colocando cada elemento na posição correspondente ao seu dígito;
   - copia `output` de volta para `v`.

O contador `num_swaps` é incrementado a cada escrita em `output` e a cada cópia de volta para `v`. O contador `num_comps` praticamente não é usado nessa implementação (não há `(*num_comps)++` dentro das passagens por dígitos).

Comparações

Na prática, nesta versão:

- O número de **comparações contabilizadas** por `num_comps` é praticamente nulo (só poderia haver algo na busca do máximo, mas o código não incrementa o contador ali).
- Do ponto de vista teórico, cada Counting Sort por dígito faz apenas operações de acesso e contagem, sem comparações entre elementos, e o custo lógico por dígito é proporcional a $(n + 10)$.
- **Melhor, médio e pior caso:**

em termos de “passos elementares”, onde (d) é o número de dígitos do maior elemento.

Trocas (escritas contadas)

No código, cada escrita em `output` e cada cópia de volta para `v` conta como uma “troca”:

- Para cada dígito:
  - laço que escreve em `output`: (n) escritas → (n) incrementos em `num_swaps`;
  - laço que copia `output` para `v`: mais (n) escritas → (n) incrementos em `num_swaps`.
- Total por dígito: ($\approx 2n$) “trocas”.
- Para (d) dígitos:

  $$\text{Trocas} \approx 2dn = O(dn)$$

Complexidade de tempo

Seja (d) o número de dígitos do maior valor e a base = 10:

- Cada Counting Sort por dígito custa ($O(n + 10) \approx O(n)$).
- São feitas (d) passagens (uma por dígito):

$$T(n) = O(d \cdot n)$$

Se considerarmos (d) limitado (por exemplo, inteiros de 32 bits), isso se comporta como **($O(n)$)** na prática. Mais geralmente, pode-se escrever ($O(n \log_{10} K)$), onde (K) é o maior valor.

- **Melhor, médio e pior caso:** mesma ordem: ($O(dn)$).

Complexidade de espaço

São utilizados:

- um vetor auxiliar `output` de tamanho (n);
- um vetor `count[10]` de tamanho constante.

Logo, o espaço extra é $O(n)$

Em resumo, este Radix Sort com Counting Sort estável por dígito apresenta tempo ($O(dn)$) em qualquer caso, com número de “trocas” proporcional a (dn) e custo de memória adicional ($O(n)$), sendo muito eficiente para inteiros com número de dígitos limitado.

3. Decisões de implementação

Nessa seção, discutiremos as decisões de implementação do nosso projeto, explicando o porquê da existência das funções utilizadas no código e do modo como elas foram implementadas. Essa discussão será feita separadamente para as funções auxiliares e as funções dos algoritmos de ordenação em si.

3.1 Funções Auxiliares

- `swap`**:** Esta função simplesmente realiza a troca de dois valores. Usado em vários métodos de ordenação para a troca do lugar dos elementos no vetor. Implementada para maior concisão das demais funções visto que ao longo do código várias trocas devem ser feitas. Incrementa o número de movimentações de registros internamente, recebendo o ponteiro `num_swaps` como parâmetro por referência.

- `imprimir_prompt` : Imprime as opções de algoritmos de ordenação e lê o algoritmo escolhido pelo usuário, o tamanho do vetor a ser ordenado e o estado inicial do vetor(aleatório, ordenado, inversamente ordenado). Essas informações são salvas em variáveis separadas e serão utilizadas por outras funções para a construção e ordenação do vetor.
- `gerar_aleatorio` : Recebe como argumento um vetor de inteiros de tamanho n definido pelo usuário, e atribui, para cada elemento do vetor, um número aleatório entre 0 e $7n$. Utiliza a função `rand()` de `stdlib.h` , e limita ao intervalo definido utilizando aritmética modular.
- `gerar_inversamente` : Recebe como argumento um vetor de inteiros de tamanho n, e insere neles os números de 1 a n de maneira inversamente ordenada.
- `gerar_ordenado` : Recebe como argumento um vetor de inteiros de tamanho n, e insere neles os números de 1 a n de maneira diretamente ordenada.
- `switch_case_metodo` : Função que recebe como um de seus argumentos o algoritmo de ordenação escolhido pelo usuário, e aplica ao vetor (já no estado definido pelo usuário). Também é responsável por contar o tempo gasto para a execução de cada algoritmo, utilizando a função `clock()` da biblioteca `time.h` . Por fim, retorna ao usuário o número de comparações e o de trocas realizadas durante a execução, além do tempo gasto.

3.2 Função `main`

Na função main, são declaradas as variáveis que armazenam o tamanho do vetor, o método de ordenação, o estado inicial do vetor, o número de trocas e o número de comparações, além do vetor de inteiros em si(único alocado dinamicamente). Assim, ela executa, em ordem, as seguintes operações:

1. Lê o tamanho do vetor, o método de ordenação e o estado inicial do vetor utilizando a função `imprimir_prompt` .
2. Caso o estado do vetor seja aleatório, são criados mais 5 vetores de inteiros de mesmo tamanho n, e cada um deles é preenchido com números aleatórios utilizando `gerar_aleatorio` . Isso é feito para garantir resultados mais genéricos para vetores ordenados. Depois, a função `switch_case_metodo` é chamada 5 vezes, uma para cada vetor, ordenando assim os 5 vetores e retornando ao usuário as informações necessárias.
3. Caso o estado do vetor seja ordenado ou inversamente ordenado, o programa chama a função `gerar_ordenado` ou `gerar_inversamente` , respectivamente. Após isso, chama a função `switch_case_metodo` , e ordena corretamente o vetor.
4. Por fim, libera a memória alocada dinamicamente e encerra o programa

3.3 Implementação dos Algoritmos de Ordenação

OBS: Abaixo estarão as decisões de implementação de todos métodos de ordenação. Destacamos que todos eles possuem dois parâmetros para o controle das estatísticas (`num_comps` - quantidade de comparações de chaves; `num_swaps` - quantidade de movimentações de registros). Estes são ponteiros para inteiros, que são passados por referência para que seus valores sejam corretamente atualizados dentro de chamadas recursivas e eventuais chamadas de funções auxiliares.

- `bubblesort`**:** Implementamos o Bubble Sort em sua versão aprimorada para obter maior eficiência neste algoritmo de ordenação. O uso do aprimoramento deste método faz com que ele encerre seu funcionamento caso, em alguma iteração, ele percorra o vetor inteiro e não realize nenhuma troca. O número de comparações é incrementado a cada comparação de elementos adjacentes, e o de movimentações a cada chamada da função `swap`, internamente.
- `selectionsort`**:** O Selection Sort seleciona um elemento i que vai de 0 a $n-1$ como o menor e percorre o vetor comparando este menor aos demais elementos. Caso ele encontre algum número menor,

seleciona este. Ao final de cada iteração, coloca o menor elemento no índice correto i. Deste modo, nosso código incremente o número de comparações ao comparar o menor elemento atual com os demais, e incremento as movimentações de registros ao posicionar o menor elemento no índice correto, por meio da função `swap`.

- `insertionsort`: No Insertion Sort, as comparações são computadas ao comparara o elemento ($v[i]$) com os elemento anteriores a eles no vetor, até encontrar sua posição correta. A quantidade de movimentações é incrementada a cada desocamento dos elementos para o índice a frente, assim como na inserção do elemento $v[i]$ no posição correta.

- `shellsort`: Como o Shell Sort é apenas uma generalização do Insertion Sort, a maneira como computamos comparações e movimentações de registros é análoga àcima. Vale destacar a escolha dos incrementos, importantíssimo para tornar esse método eficiente. Os incrementos seguem a sequência de Knuth, em que:
  $h_{k+1} = 3 \cdot h_k + 1$
  Limitamos o vetor de incrementos a 10 índices, logo, o maior incremento possível equivale ao décimo termo da sequência. A quantidade de incrementos também é limitado pelo tamanho do vetor que está sendo ordenado, de modo que os incrementos não podem superar esse número (ex: para $n = 10$, os incrementos serão apenas [1, 4], pois o próximo termo (13) supera n). Assim, os incrementos são de $1, 4, 13, 40, 121$... Eles são, porém invertidos pois o Shell Sort deve realizar o insertion sort com intervalos decrescentes.

- `quicksort`: Implementamos nosso algoritmo de Quick Sort com partição interna (não há uma função para particionar o vetor) e usamos a função `insertionsort` para realizar a escolha do pivô como mediana de 3: dentro do `quicksort` é feita uma chamada ao insertion com três elementos (o primeiro, o do meio e o do final), em que o elemento com valor intermediário é selecionado como pivô. Nesta etapa os números de comparações e movimentações também são incrementados, de acordo com a lógica do insertion acima.
  As comparações são computadas ao verificar se os elementos da posição i são maior que o pivo ou se os elementos da posição j são menores. A quantidade de movimentações de registro é incrementada pela função `swap` ao chamá-la para os elementos i e j.

- `heapify`: Esta função é responsável por transformar o vetor em uma heap máxima. Neste processo, seleciona-se um índice como o maior entre os demais e os dois elementos que ficarão à sua esquerda e direita. Compara-se, então, o elemento da esquerda e o da direita com o atual maior, momento em que o maior é atualizado caso ele não seja, de fato, o maior, e também momento em que o número de comparações é incrementado. Caso o elemento originalmente escolhido como maior não seja, realmente, o maior, troca ele de lugar com o verdadeiro maior pela função `swap`, incrementando o número de movimentações.

- `heapsort`: O Heap Sort ordena por meio da criação de heaps máximas a partir do último nó pai até a raiz. Após encontrar a raiz da heap (maior elemento), entra-se num loop em que coloca-se a raiz na posição correta ($i - 1$, inicializando $i = n$ e decrementando-o) por meio da função `swap` (incrementando a quantidade de movimentações de registro) e chamando o `heapify` novamente.

- `merge`: Esta função recebe um vetor de tamanho n e divide-o em 2 vetores de tamanho $n/2$, um da esquerda e um da direita, a partir do meio. Em sequência, compara os elemento do vetor da esquerda e da direita, inserindo os menores dos dois números sendo comparados no vetor. A cada execução desse processo o número de comparações de chaves é incrementado, assim como o de movimentações.

- `mergesort`: O Merge Sort é um algoritmo de divisão e conquista que divide o vetor de tamanho n em n vetores unitários e depois ordena-os. Deste modo, esta função apenas realiza a partição do array em arrays unitários que estão garantidamente ordenados e, após isso, ordena-o chamando a função

`merge`, logo, não manipula o número de comparações e movimenações, tendo em vista que isso é feito na função acima.

- `ranksort`: Compara um elemento de índice i com os demais elementos do vetor procurando por elementos menores ou iguais a ele, porém de índice diferente, para a criação do vetor auxiliar. Neste processo, incrementa a quantidade de comparações de modo que, ao final, são feitas n^2 comparações. O próximo passo é a passagem dos elementos do vetor inicial para o vetor ordenado com a ajuda do vetor auxiliar, momento em que são realizadas n movimentações. Por fim, copia-se o vetor ordenado de volta para o vetor original incrementando o número de movimentações mais n vezes.
- `radixsort`: Foi implementado utilizando uma função auxiliar `countingSortForRadix`, função essa que ordena os números por dígito(preservando a ordem original para números com dígitos iguais). Depois dessa ordenação, os números são devolvidos para o vetor original, e depois o processo se repete até o último dígito. Precisa percorrer o vetor uma vez por inteiro, para descobrir o maior número de dígitos(no maior número). Esse é o único momento em que ocorrem comparações(n-1) entre elementos.

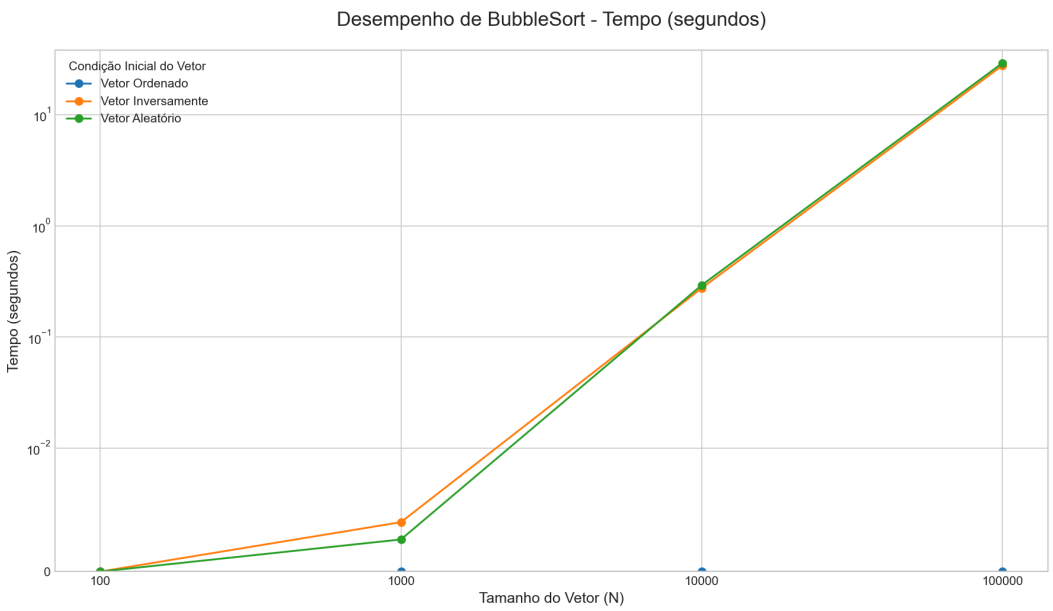
4. Resultados

Abaixo, estão alguns resultados relativos à execução dos algoritmos de ordenação, considerando diferentes estados iniciais de vetores e métodos de ordenação. Cada gráfico compara alguma das métricas analisadas(tempo de execução, número de trocas ou número de comparações) para os quatro tamanhos de vetor(100, 1000, 10000 e 100000 elementos) e para os três estados do vetor(ordenado, inversamente ordenado, aleatório).

4.1 BubbleSort

4.1.1 Análise temporal

Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.

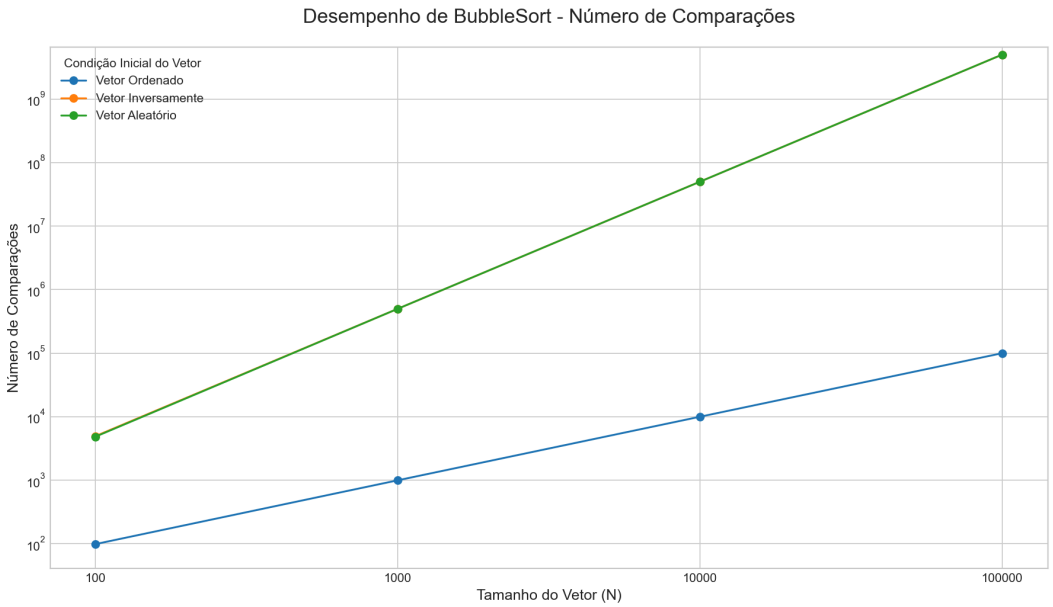


Observamos um crescimento quadrático para o tempo de execução, assim como esperado na análise teórica. O tempo de execução fica praticamente zero(considerando uma análise com 7 casas decimais de

precisão) para um vetor ordenado, pois a implementação com aprimoramento faz com que só seja necessário percorrer o vetor uma vez nesse caso)

4.1.2 Análise de comparações

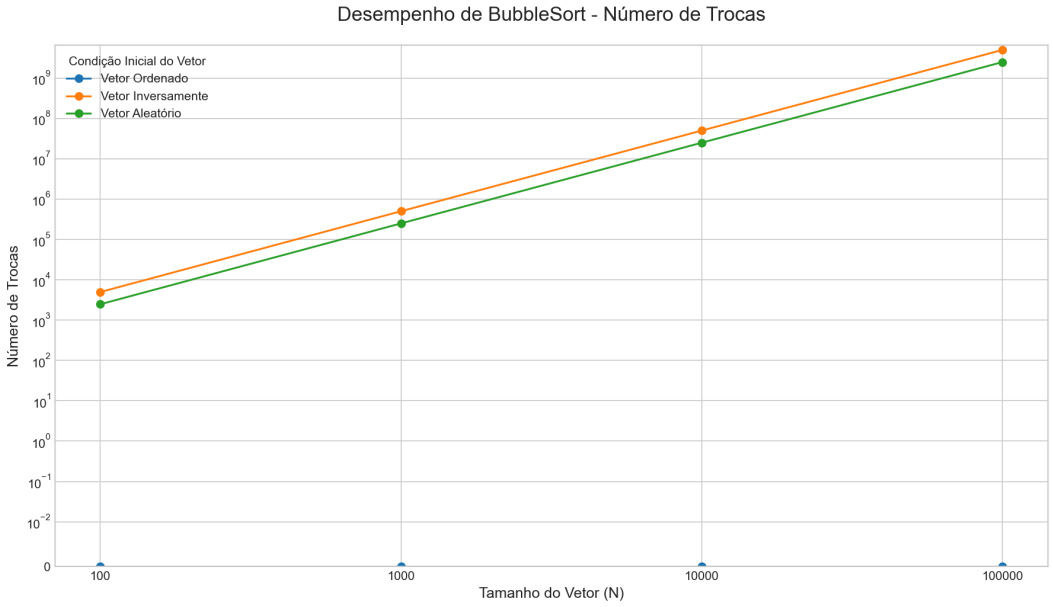
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



No gráfico, o número de comparações para vetores aleatórios é praticamente o mesmo que do vetor ordenado inversamente, o que faz com que as linhas se sobreponham. Para vetores ordenados, o número de comparações é exatamente igual ao tamanho do vetor, o que é justificado pelo aprimoramento.

4.1.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

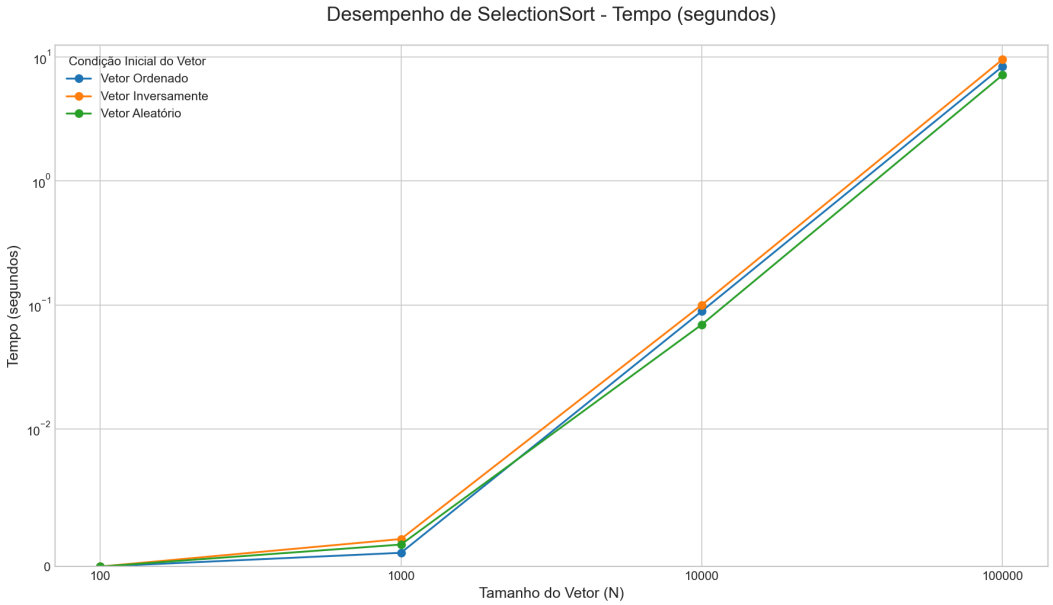


Novamente, o vetor inversamente ordenado e os vetores aleatórios apresentam número de trocas de mesma grandeza para um mesmo tamanho de vetor. O vetor ordenado inversamente exige, contudo, uma quantidade ligeiramente maior de trocas. Para vetores ordenados, o número de trocas será sempre 0.

4.2 SelectionSort

4.2.1 Análise temporal

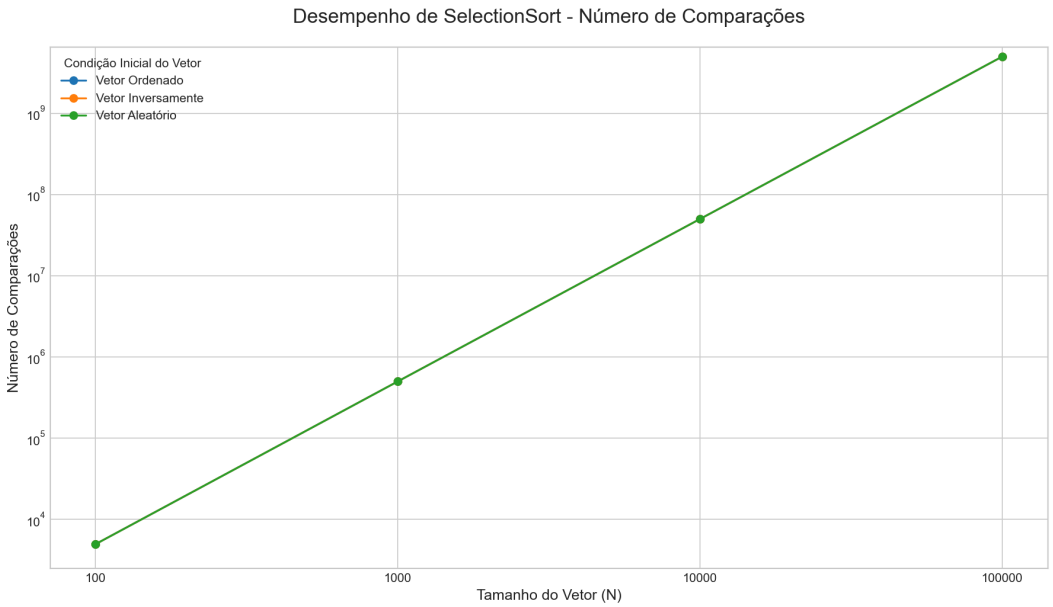
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Para esse algoritmo, fica evidente o tempo de execução quadrático para todos os estados do vetor.

4.2.2 Análise de comparações

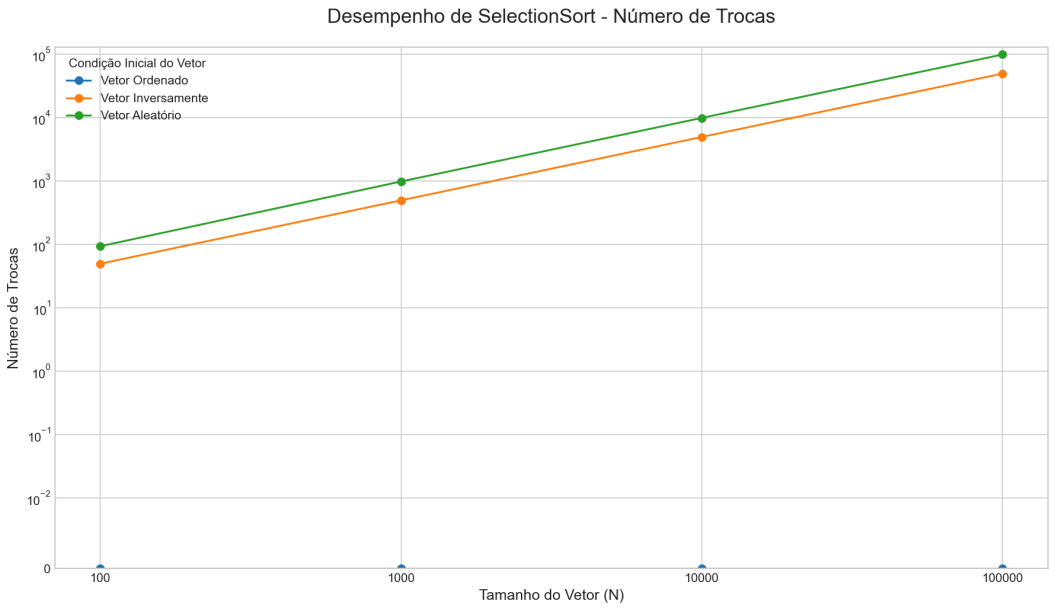
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



O SelectionSort realiza o mesmo número de comparações para determinado tamanho de vetor, independentemente do estado do mesmo.

4.2.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

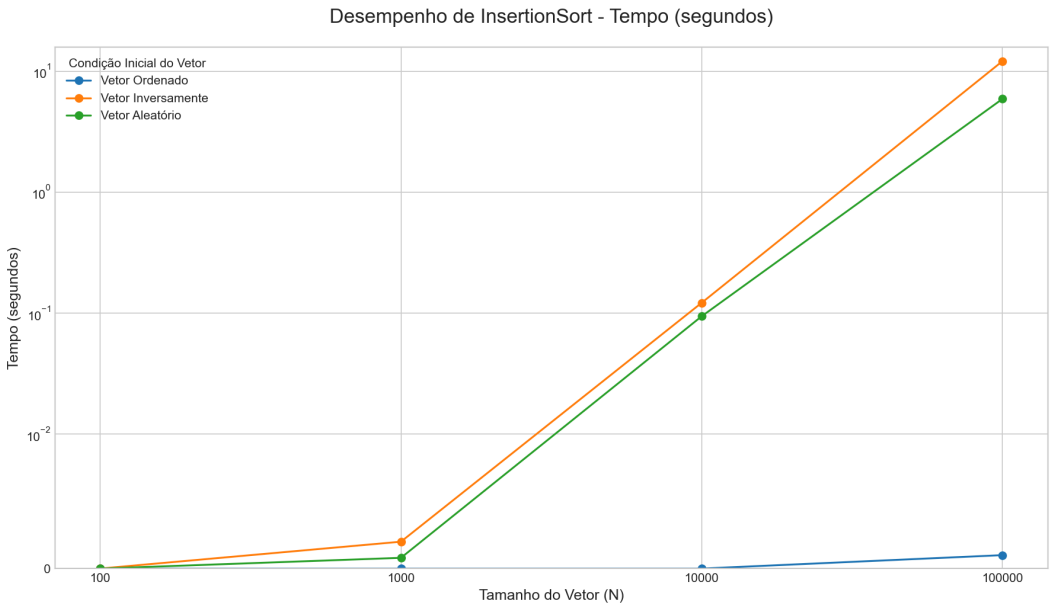


Pelos dados, o vetor inversamente ordenado exige um número ligeiramente menor de trocas que os vetores aleatórios.

4.3 InsertionSort

4.3.1 Análise temporal

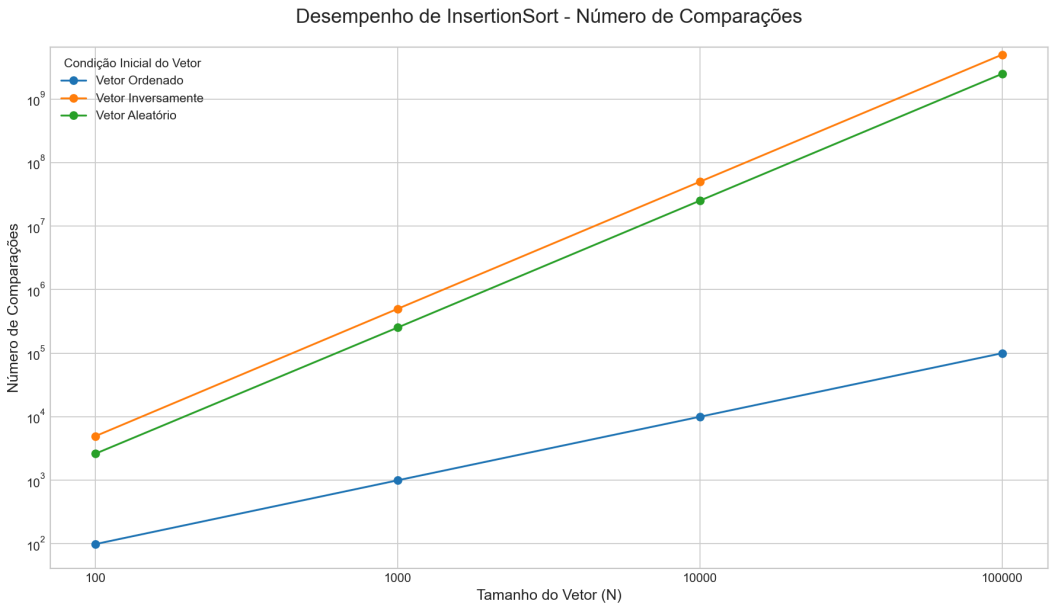
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



O custo quadrático do caso médio e do pior caso ficam evidentes no gráfico do vetor inversamente ordenado e dos vetores aleatórios. Além disso, também fica evidente o bom melhor caso desse algoritmo(vetor ordenado), que exige tempo de execução baixíssimo.

4.3.2 Análise de comparações

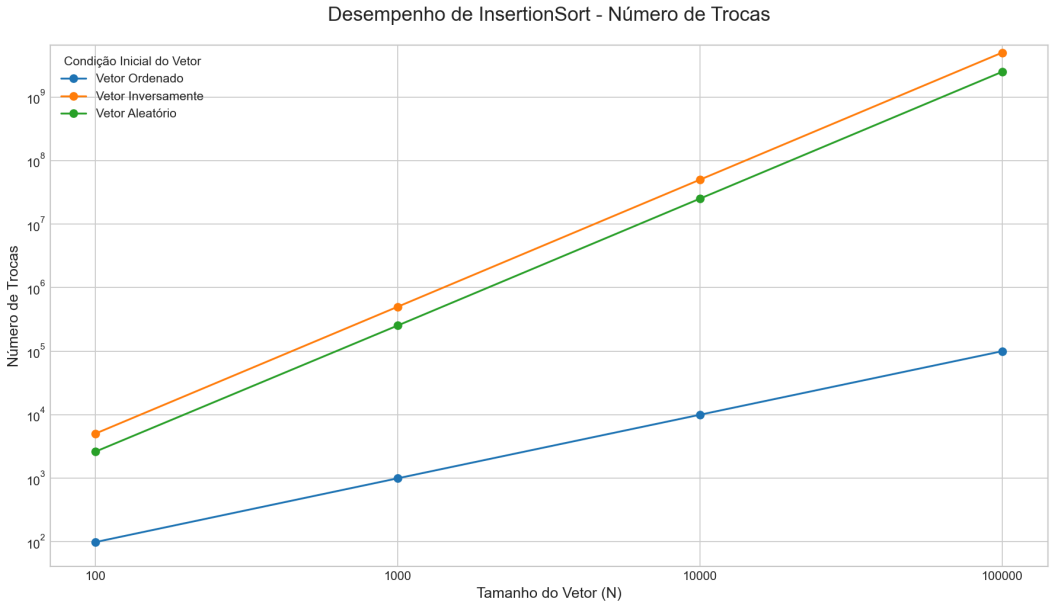
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Vetores inversamente ordenados e aleatórios exigem quantidades parecidas de comparações, enquanto vetores ordenados exigem sempre n-1 comparações

4.3.3 Análise de trocas

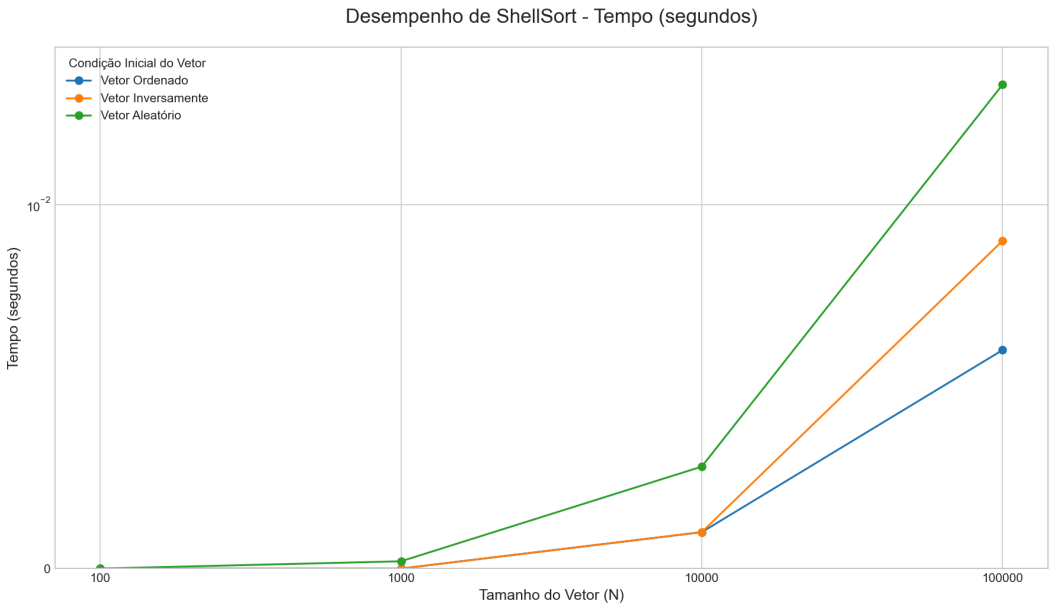
Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.



4.4 ShellSort

4.4.1 Análise temporal

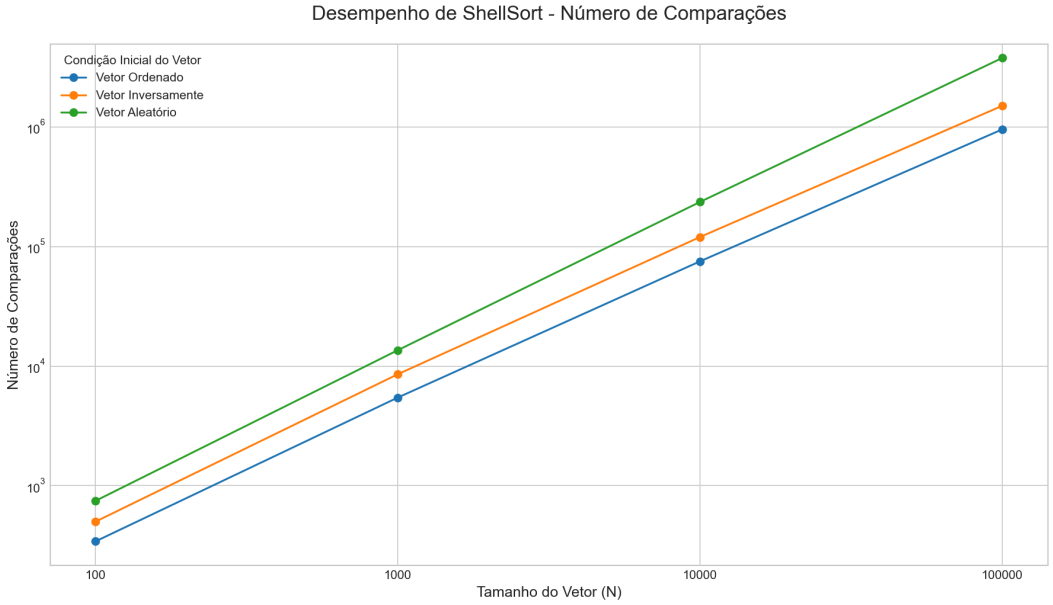
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Primeiro algoritmo até agora com custo menor que quadrático. Vemos como os tempos de execução são menores para todas as configurações do vetor. Vetores ordenados e inversamente ordenados possuem desempenho ligeiramente melhor

4.4.2 Análise de comparações

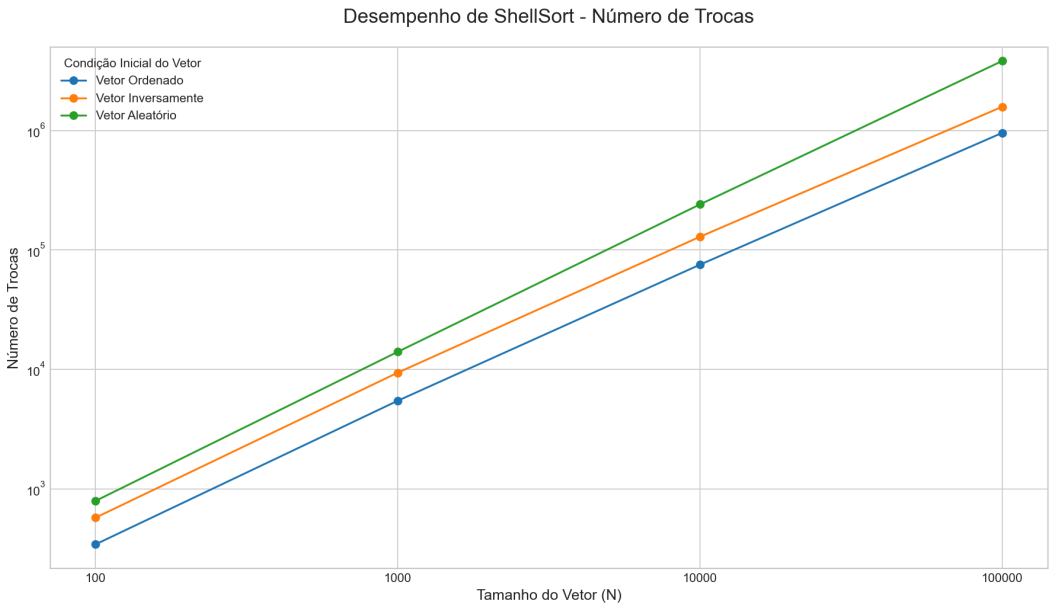
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Número de comparações é maior para vetores aleatórios e menor para vetores ordenados, com vetores inversamente ordenados sendo intermediários.

4.4.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

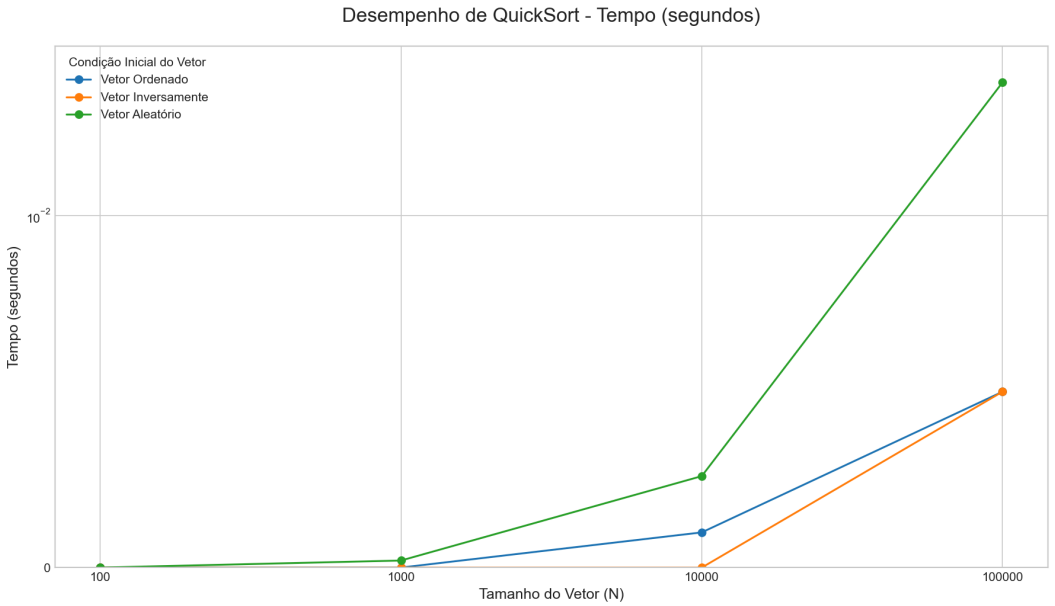


Comportamento do número de trocas semelhante ao do número de comparações.

4.5 QuickSort

4.5.1 Análise temporal

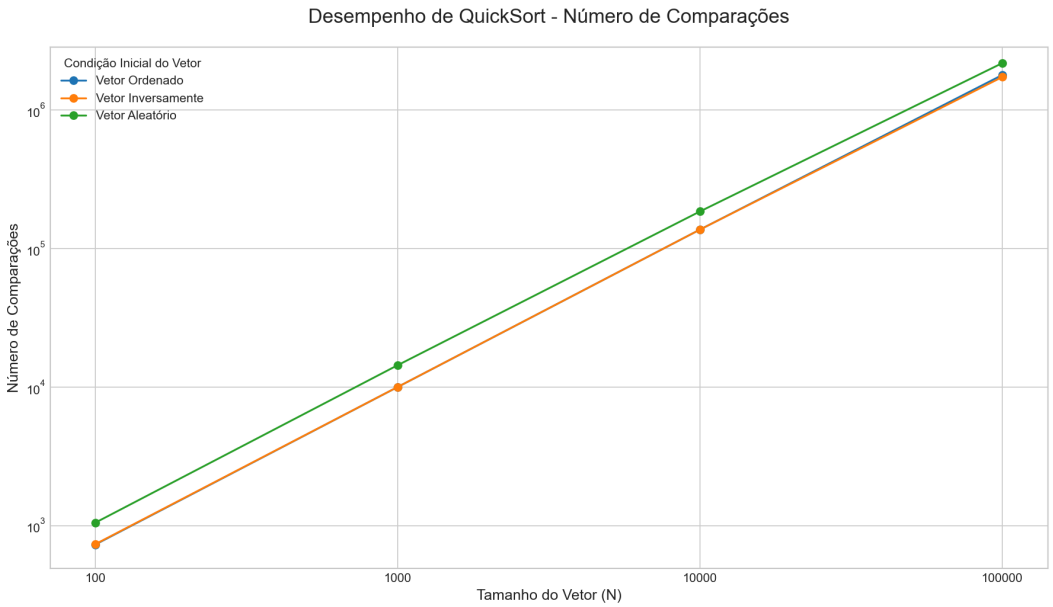
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Algoritmo mais rápido até agora dentre os apresentados. Tempo de execução piora para vetores aleatórios, mas ainda assim supera os outros algoritmos.

4.5.2 Análise de comparações

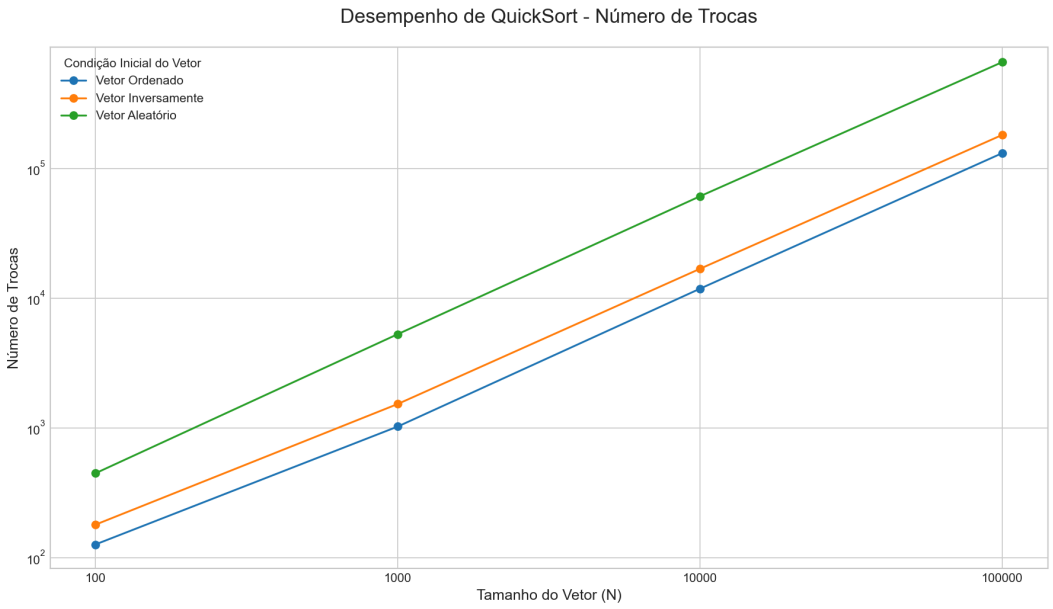
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Número de comparações extremamente semelhante para vetores ordenados e inversamente ordenados(causando sobreposição nos gráficos), com número para vetores aleatórios ligeiramente maior.

4.5.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

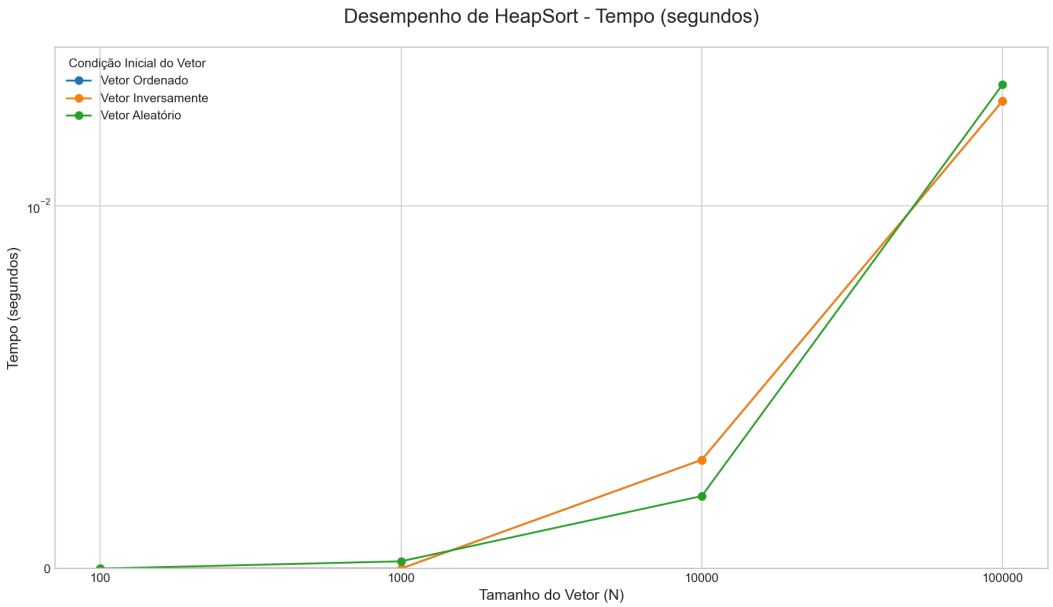


Menos trocas executadas para vetores ordenados e inversamente ordenados quando comparadas com vetores aleatórios.

4.6 HeapSort

4.6.1 Análise temporal

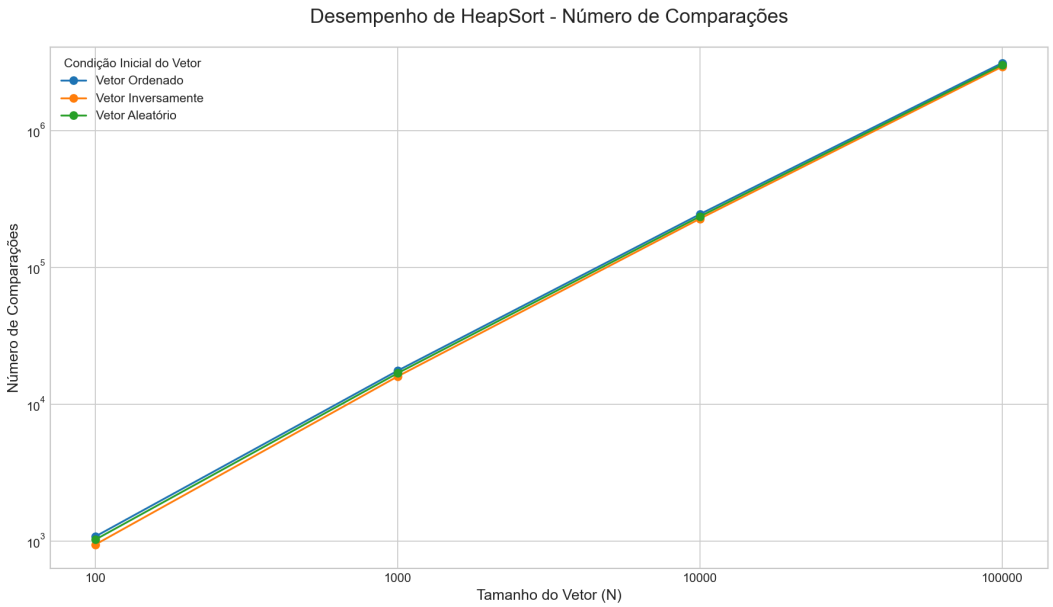
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Esse gráfico deixa evidente o custo constante O(n*log(n)) do HeapSort, em que todos os casos apresentam desempenho extremamente semelhante

4.6.2 Análise de comparações

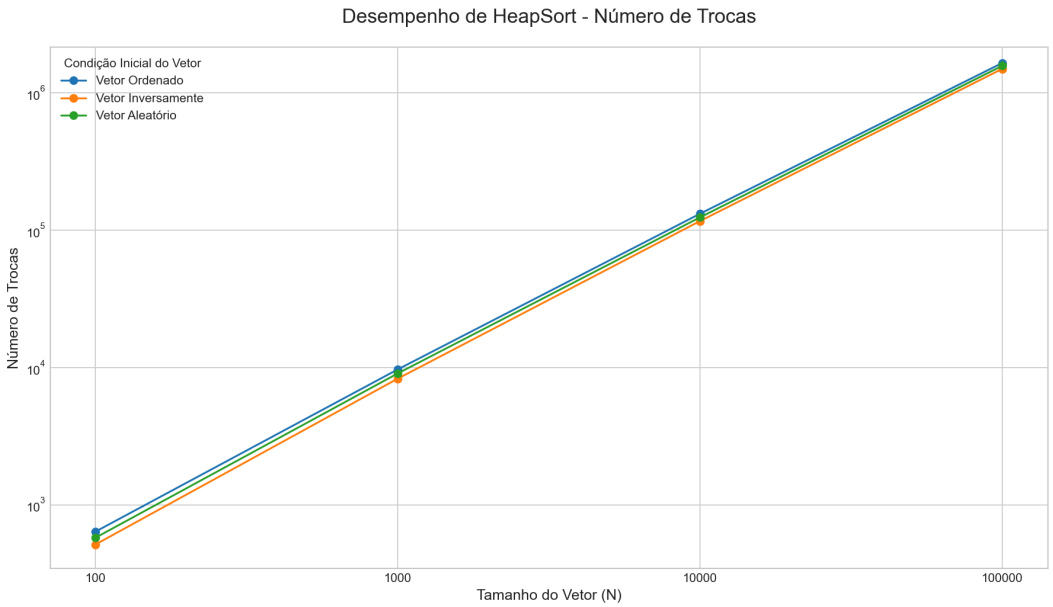
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Número de comparações semelhantes em todos os casos. Nesse algoritmo, elas provém das operações de montar a Heap e de reconstrui-la toda vez que o elemento no topo é retirado e considerado ordenado

4.6.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

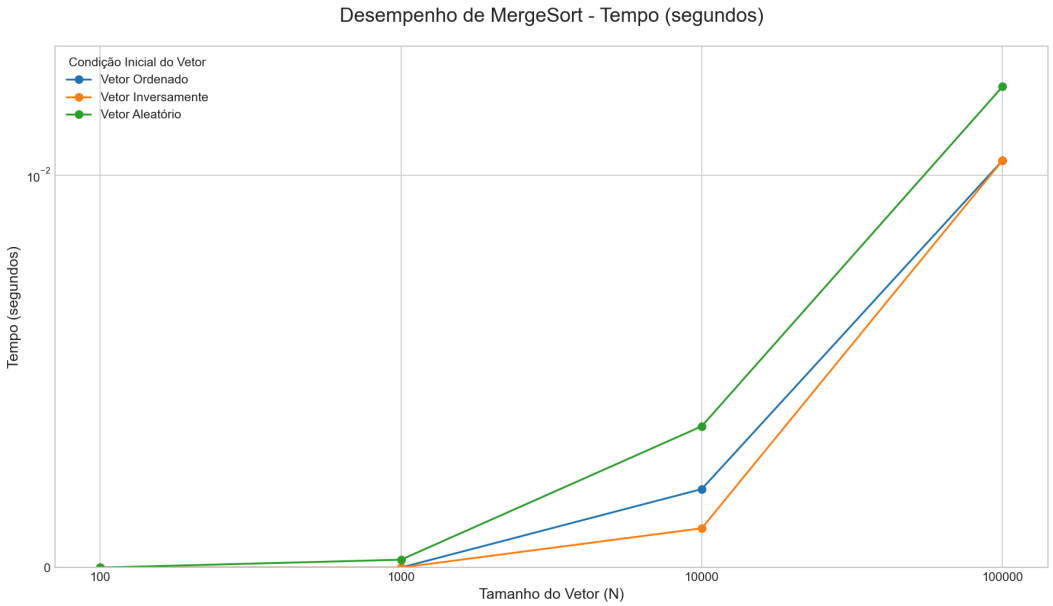


Trocas também semelhantes em todos os casos, provindo da montagem e manutenção da Heap.

4.7 MergeSort

4.7.1 Análise temporal

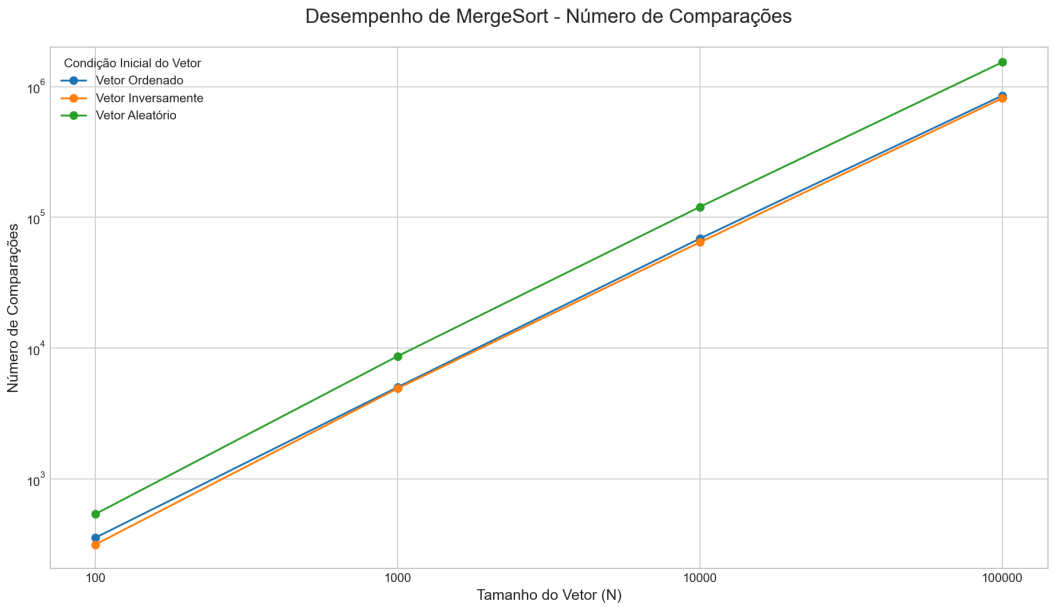
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Complexidade temporal $O(n * log(n))$ novamente fica evidente para todos os casos.

4.7.2 Análise de comparações

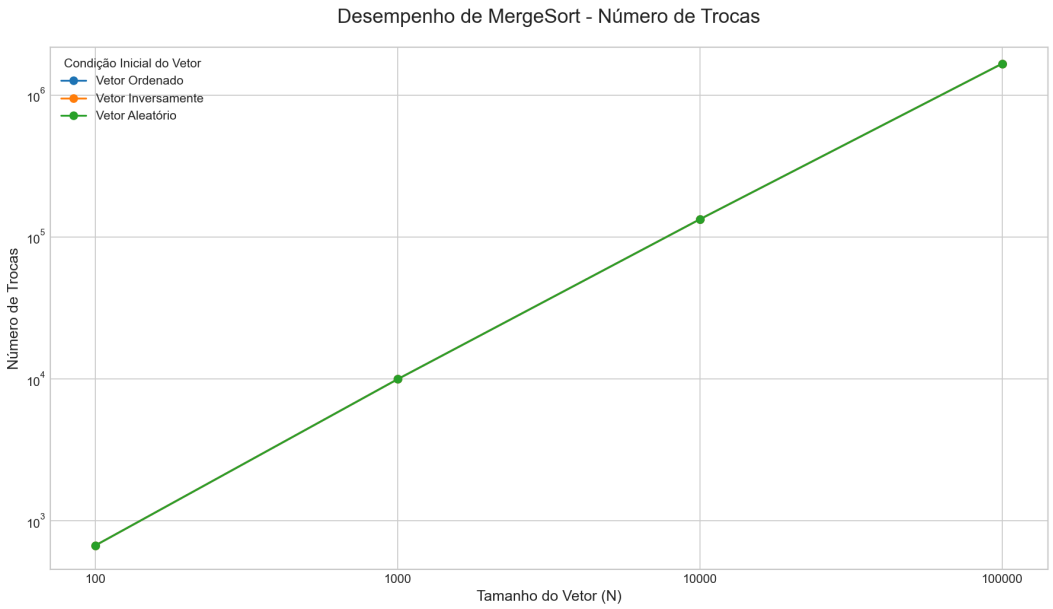
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Quantidade de comparações semelhantes em todos os casos

4.7.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

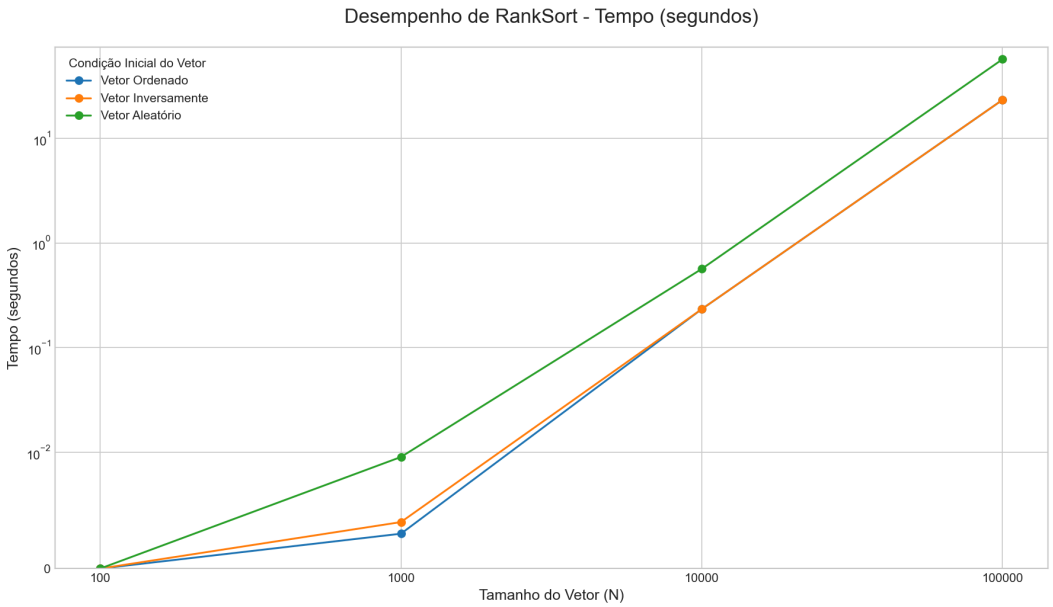


Número de trocas constante para um vetor de mesmo tamanho.

4.8 Contagem de Menores

4.8.1 Análise temporal

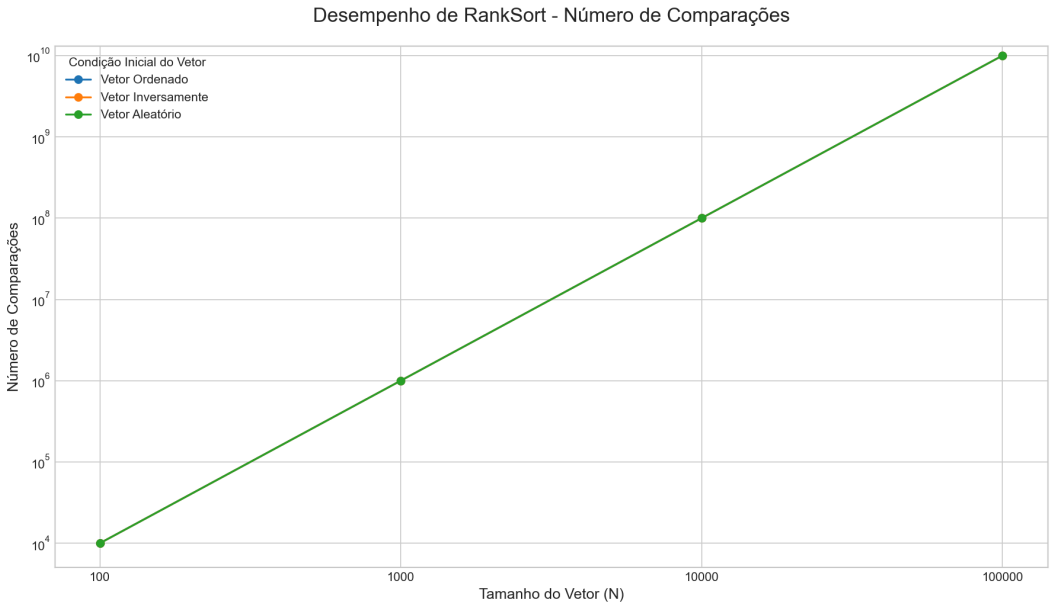
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Algoritmo com complexidade quadrática para todos os casos, o que fica evidente no gráfico. Ligeiramente pior para vetores aleatórios

4.8.2 Análise de comparações

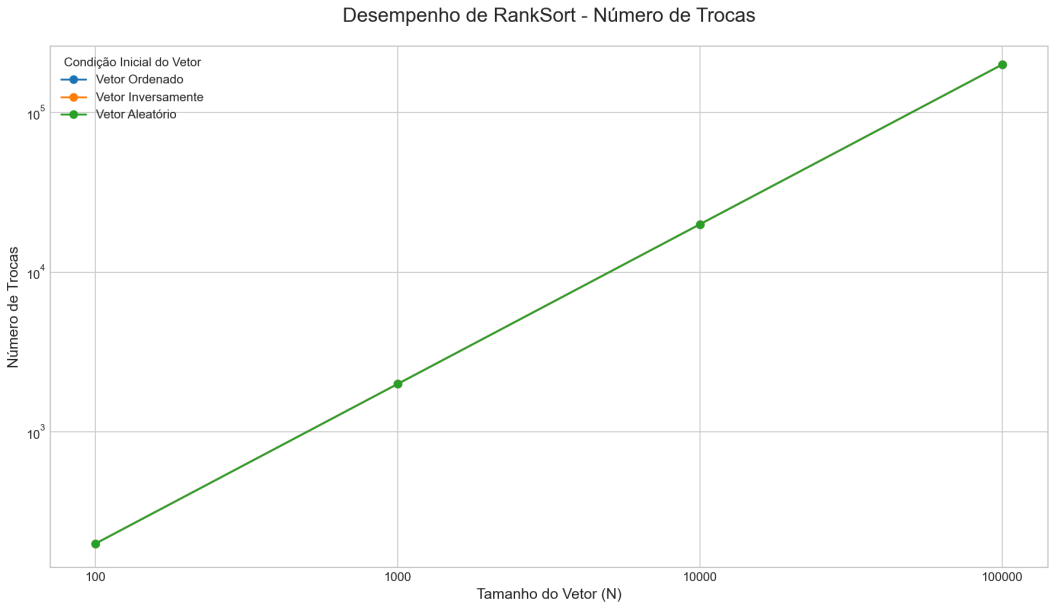
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Número de comparações constante para determinado tamanho de vetor.

4.8.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.

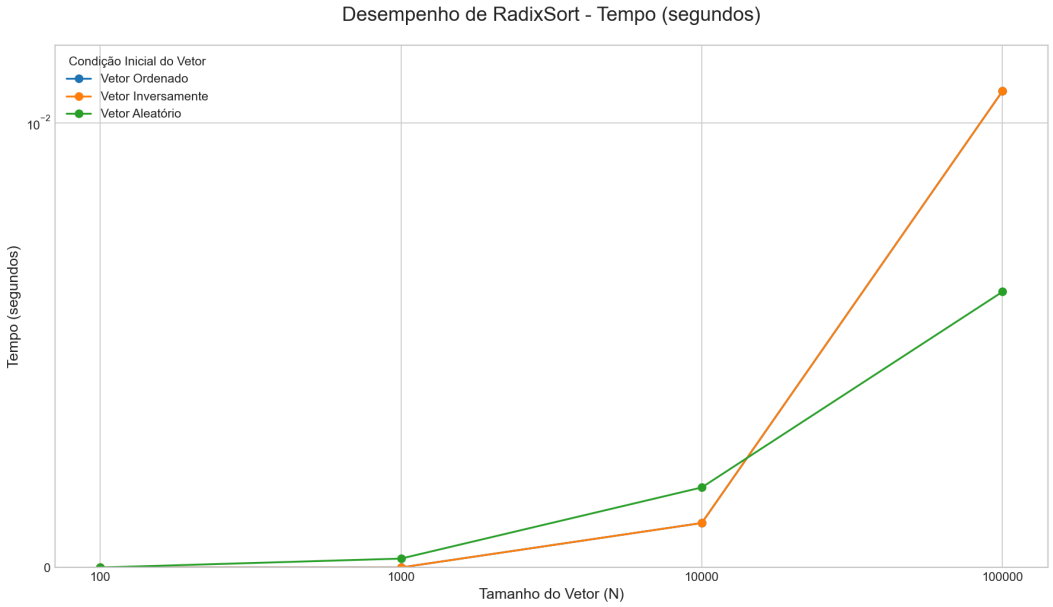


Número de trocas constante para determinado tamanho de vetor

4.9 RadixSort

4.9.1 Análise temporal

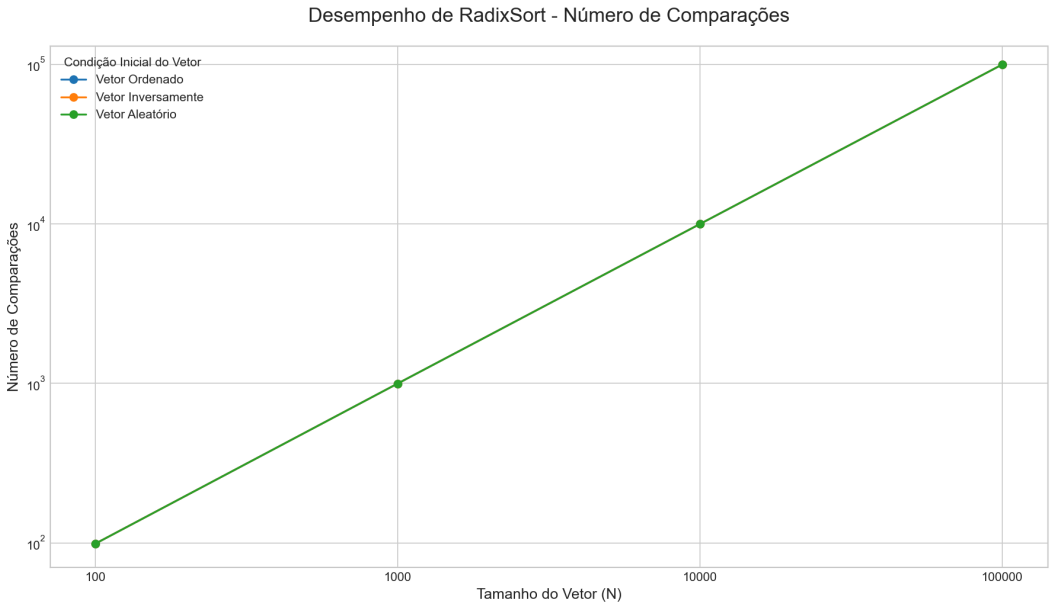
Abaixo, está a análise temporal para esse algoritmo, de acordo com os tamanhos de entrada.



Único algoritmo apresentado neste projeto que possui complexidade melhor que O(n*log(n)). Desempenho temporal igual para vetores ordenados e inversamente ordenados, e ligeiramente pior para vetores aleatórios.

4.9.2 Análise de comparações

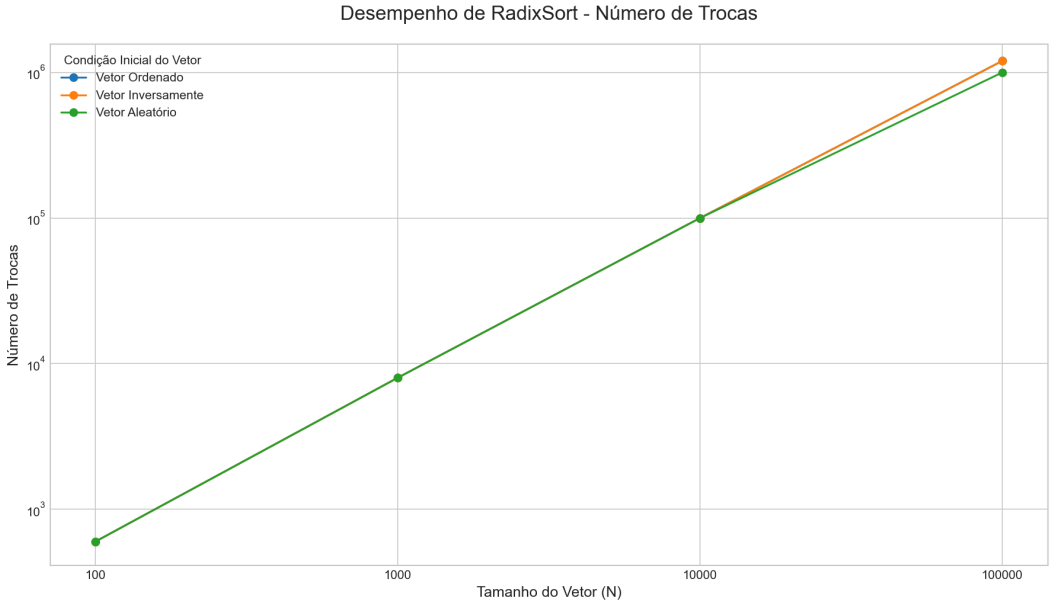
Abaixo, está a análise da quantidade de comparações para esse algoritmo, de acordo com os tamanhos de entrada.



Número de comparações constante e igual a n-1. Ocorre por conta da implementação, que exige que o maior número do vetor seja encontrado para determinar a quantidade máxima de dígitos(o algoritmo é, por definição, não-comparativo).

4.9.3 Análise de trocas

Abaixo, está a análise da quantidade de trocas para esse algoritmo, de acordo com os tamanhos de entrada.



Número de trocas praticamente constante para todos os casos. Ocorre na transferência entre vetores.

4.10 Tabela Geral de Resultados

dados.csv

Algoritmo	Vetor	Tamanho	Comparaçoes	Trocas	Tempo(s)
BubbleSort	Ordenado	100	99	0	0
BubbleSort	Ordenado	1000	999	0	0
BubbleSort	Ordenado	10000	9999	0	0
BubbleSort	Ordenado	100000	99999	0	0
BubbleSort	Inversamente	100	4950	4950	0
BubbleSort	Inversamente	1000	499500	499500	0,004
BubbleSort	Inversamente	10000	49995000	49995000	0,275
BubbleSort	Inversamente	100000	4999950000	4999950000	27,733
BubbleSort	Aleatório	100	4852,6	2466	0
BubbleSort	Aleatório	1000	499237,6	249313,6	0,0026
BubbleSort	Aleatório	10000	49977029,2	25006393	0,2924
BubbleSort	Aleatório	100000	4999852745	2499623803	29,158
SelectionSort	Ordenado	100	4950	0	0
SelectionSort	Ordenado	1000	499500	0	0,001
SelectionSort	Ordenado	10000	49995000	0	0,089
SelectionSort	Ordenado	100000	4999950000	0	8,405
SelectionSort	Inversamente	100	4950	50	0
SelectionSort	Inversamente	1000	499500	500	0,002
SelectionSort	Inversamente	10000	49995000	5000	0,1
SelectionSort	Inversamente	100000	4999950000	50000	9,567
SelectionSort	Aleatório	100	4950	94,4	0
SelectionSort	Aleatório	1000	499500	991,4	0,0016
SelectionSort	Aleatório	10000	49995000	9989,6	0,0694
SelectionSort	Aleatório	100000	4999950000	99987,2	7,1656
InsertionSort	Ordenado	100	99	99	0
InsertionSort	Ordenado	1000	999	999	0
InsertionSort	Ordenado	10000	9999	9999	0
InsertionSort	Ordenado	100000	99999	99999	0.001
InsertionSort	Inversamente	100	4950	5049	0
InsertionSort	Inversamente	1000	499500	500499	0,002

Algoritmo	Vetor	# Tamanho	Comparaçoes	Trocas	Tempo(s)
InsertionSort	Inversamente	10000	49995000	50004999	0,122
InsertionSort	Inversamente	100000	4999950000	5000049999	12,131
InsertionSort	Aleatório	100	2622,2	2625,8	0
InsertionSort	Aleatório	1000	253513,8	253520,2	0,0008
InsertionSort	Aleatório	10000	25160194,6	25160203,2	0,0944
InsertionSort	Aleatório	100000	2494009410	2494009420	5,9124
ShellSort	Ordenado	100	342	342	0
ShellSort	Ordenado	1000	5457	5457	0
ShellSort	Ordenado	10000	75243	75243	0,001
ShellSort	Ordenado	100000	955719	955719	0,007
ShellSort	Inversamente	100	500	572	0
ShellSort	Inversamente	1000	8550	9377	0
ShellSort	Inversamente	10000	120190	128947	0,001
ShellSort	Inversamente	100000	1504850	1581007	0,011
ShellSort	Aleatório	100	746,2	790,2	0
ShellSort	Aleatório	1000	13581,8	14010,6	0,0002
ShellSort	Aleatório	10000	236208,8	240553,4	0,0028
ShellSort	Aleatório	100000	3781238,8	3822157,6	0,0232
QuickSort	Ordenado	100	732	126	0
QuickSort	Ordenado	1000	10031	1022	0
QuickSort	Ordenado	10000	137247	11808	0,001
QuickSort	Ordenado	100000	1791086	131070	0,005
QuickSort	Inversamente	100	737	179	0
QuickSort	Inversamente	1000	10039	1525	0
QuickSort	Inversamente	10000	137263	16813	0
QuickSort	Inversamente	100000	1731101	181073	0,005
QuickSort	Aleatório	100	1053	445,8	0
QuickSort	Aleatório	1000	14414,6	5267,2	0,0002
QuickSort	Aleatório	10000	185511,2	60906,4	0,0026
QuickSort	Aleatório	100000	2181463	659243,6	0,015
HeapSort	Ordenado	100	1081	640	0
HeapSort	Ordenado	1000	17583	9708	0

Algoritmo	Vetor	# Tamanho	Comparaçoes	Trocas	Tempo(s)
HeapSort	Ordenado	10000	244460	131956	0,003
HeapSort	Ordenado	100000	3112517	1650854	0,035
HeapSort	Inversamente	100	944	516	0
HeapSort	Inversamente	1000	15965	8316	0
HeapSort	Inversamente	10000	226682	116696	0,003
HeapSort	Inversamente	100000	2926640	1497434	0,021
HeapSort	Aleatório	100	1025,6	577,8	0
HeapSort	Aleatório	1000	16851,8	9078	0,0002
HeapSort	Aleatório	10000	235389,8	124203,4	0,002
HeapSort	Aleatório	100000	3019621,8	1575034,6	0,0236
MergeSort	Ordenado	100	356	672	0
MergeSort	Ordenado	1000	5044	9976	0
MergeSort	Ordenado	10000	69008	133616	0,002
MergeSort	Ordenado	100000	853904	1668928	0,019
MergeSort	Inversamente	100	316	672	0
MergeSort	Inversamente	1000	4932	9976	0
MergeSort	Inversamente	10000	64608	133616	0,001
MergeSort	Inversamente	100000	815024	1668928	0,014
MergeSort	Aleatório	100	540,4	672	0
MergeSort	Aleatório	1000	8702	9976	0,0002
MergeSort	Aleatório	10000	120477,4	133616	0,0036
MergeSort	Aleatório	100000	1536233,8	1668928	0,0178
RankSort	Ordenado	100	10000	200	0
RankSort	Ordenado	1000	1000000	2000	0,003
RankSort	Ordenado	10000	100000000	20000	0,234
RankSort	Ordenado	100000	10000000000	200000	23,411
RankSort	Inversamente	100	10000	200	0
RankSort	Inversamente	1000	1000000	2000	0,004
RankSort	Inversamente	10000	100000000	20000	0,234
RankSort	Inversamente	100000	10000000000	200000	23,411
RankSort	Aleatório	100	10000	200	0
RankSort	Aleatório	1000	1000000	2000	0,0096

Algoritmo	Vetor	# Tamanho	Comparaçoes	Trocas	Tempo(s)
RankSort	Aleatório	10000	100000000	20000	0,567
RankSort	Aleatório	100000	10000000000	200000	57,2646
RadixSort	Ordenado	100	99	600	0
RadixSort	Ordenado	1000	999	8000	0
RadixSort	Ordenado	10000	9999	100000	0,001
RadixSort	Ordenado	100000	99999	1200000	0,012
RadixSort	Inversamente	100	99	600	0
RadixSort	Inversamente	1000	999	8000	0
RadixSort	Inversamente	10000	9999	100000	0,001
RadixSort	Inversamente	100000	99999	1200000	0,012
RadixSort	Aleatório	100	99	600	0
RadixSort	Aleatório	1000	999	8000	0,0002
RadixSort	Aleatório	10000	9999	100000	0,0018
RadixSort	Aleatório	100000	99999	1200000	0,0062

5. Análise comparativa dos Algoritmos

Para essa análise, compararemos os desempenhos dos algoritmos separados de grupo, divididos de acordo com sua complexidade temporal. Faremos a análise comparativa em relação às comparações, às movimentações de registros e ao tempo de execução, para os 3 possíveis estados originais do vetor.

5.1 Grupo quadrático - $O(n^2)$

Nesse grupo estão Bubble Sort, Selection Sort, Insertion Sort e Rank Sort (Contagem de Menores). Todos eles tem crescimento assintótico quadrático, implicando que tenham pior desempenho conforme n aumenta.

5.1.1 Análise em Relação às Quantidades de Comparações de Chaves

OBS: O Selection Sort, independentemente do estado original do vetor, realiza sempre $\frac{n(n-1)}{2}$ comparações de chaves dado um n. Por exemplo, para $n = 100$, sempre são feitas 4950 comparações de chaves (equivalente a $\frac{n(n-1)}{2} = \frac{100 \cdot 99}{2} = 4950$). A Contagem de Menores também possui este comportamento: para dado valor de n realiza um número constante de comparações. No caso do Rank Sort, sempre são feitas n^2 comparações, logo, para $n = 100$, são feitas sempre $n^2 = 10000$ comparações.
Deste modo, não iremos discutir estes métodos abaixo.

- **Vetores Ordenados e Inversamente Ordenados**
  Para os casos em que o vetor está originalmente ordenado ou inversamente ordenado, os algoritmos Bubble Sort e Insertion Sort possuem o mesmo comportamento em relação às comparações, isto é, sempre resultam no mesmo número de comparações. Ambos são igualmente mais eficientes para

vetores ordenados em relação ao Selection e ao Rank Sort, porém desempenham equivalentemente ao primeiro destes para o vetor inversamente ordenado.

- **Vetores Aleatórios**
  O Insertion Sort prova ser o algoritmo quadrático mais eficiente no quesito comparações ao analisarmos os comportamentos para vetores aleatórios. Este algoritmo realiza, em média, cerca da metade do número de comparações do segundo melhor método, o Bubble Sort.

Desta maneira, entre os quadráticos, o algoritmo que menos realiza comparações é o Insertion, que vem na frente do Bubble, seguido pelo Selection e, por fim, pelo Rank Sort.

5.1.2 Análise em Relação às Quantidades de Movimentações de Registros

OBS: O Rank Sort possui comportamento conhecido para este parâmetro também, em função do valor de n. Neste caso, o número de movimentações de registros é sempre $2 \cdot n$, logo, para $n = 100000$, são feitas $2 \cdot n = 200000$ movimentações.
Portanto, não discutiremos este método abaixo.

Pela análise dos dados da Tabela Geral de Resultados, verifica-se que independentemente do estado original do vetor, o Selection Sort sempre realiza menos trocas, seguido pelo Bubble Sort e, finalmente, pelo Insertion. Desta maneira, é possível concluir que o Selection Sort é o algoritmo mais eficiente neste quesito.

5.1.3 Análise em Relação aos Tempos de Execução

- **Vetores Ordenados**
  Para este estado original de vetor o Bubble Sort, por termos implementado a versão aprimorada, é o mais eficiente no quesito temporal, uma vez que para o maior valor possível de $n = 100000$ ele simplesmente percorre este vetor uma vez e encerra seu funcionamento. Assim, seus tempos de execução são sempre considerados 0 (tempo menor do que possível de ser identificado pelo instrumento de medição de tempo utilizado). O Insertion Sort também é extremamente eficiente neste caso, com tempo que aproximam de zero para todos os valores de n exceto $n = 100000$. O Rank Sort obteve pior desempenho temporal para vetores ordenados, cerca de 3x mais lento do que o segundo mais lento, o Selection.
- **Vetores Inversamente Ordenados**
  Para vetores inversamente ordenados o algoritmo que obteve pior desempenho foi o Bubble Sort seguido pela Contagem de Menores. O melhor foi o Selection, seguido proximamente pelo Insertion.
- **Vetores Aleatórios**
  Novamente o Rank Sort obteve pior desempenho temporal médio, sendo cerca de 2x mais lento do que o segundo pior algoritmo, o Bubble Sort. O método com maior eficiência temporal média para vetores aleatórios foi o Insertion Sort, vencendo o Selection Sort por pequena margem.

Tendo em vista que obtivemos 3 algoritmos diferentes como melhores para os 3 diferentes estado iniciais do vetor, é difícil eleger algum que seja mais eficiente no quesito tempo. Podemos afirmar que a Contagem de Menores é extremamente ineficiente e que o Bubble Sort é superior somente para vetores ordenados. A disputa se torna entre o Selection e o Insertion, o último destes tendo melhor desempenho médio para vetores aleatórios.

5.2 Grupo Logarítmico - $O(n \cdot logn)$

Neste grupo se enquadram os métodos Quick Sort, Heap Sort, Merge Sort e Shell Sort. Vale destacar que o Shell Sort não é de fato $O(n \cdot log(n))$ (utilizando a seguência de incrementos de Knuth, o algoritmo se torna $O(n^{3/2})$), porém, seu desempenho temporal foi tão eficiente quanto os demais algoritmos deste grupo, portanto, o incluiremos aqui.

5.2.1 Análise em Relação às Quantidades de Comparações de Chaves

O Merge Sort foi o algoritmo que menos realizou comparações de chaves para qualquer valor de n e qualquer estado original do vetor, se destacando como o melhor nesta categoria de análise. Abaixo, faremos a comparação com os demais métodos do grupo logarítmico:

- **Vetores Ordenados e Inversamente Ordenados**
  O Shell Sort foi o algoritmo mais eficiente em quantidades de comparações para qualquer valore de n nestes estados, superando o Quick Sort e, por fim, o Heap Sort. Desta maneira, excetuando o Merge Sort, o Shell obteve maior eficiência de comparações e o Heap, a menor, para vetores ordenados ou inversamente ordenados.
- **Vetores Aleatórios**
  Para $n = 100$ e $n = 1000$, o Shell Sort é o mais eficiente em número médio de comparações em vetores aleatórios, ao passo que o Heap é o pior (Quick sendo o intermediário). Porém, para $n = 10000$ e $n = 100000$, a eficiência do Shell para comparações decai muito, tornando-o o algoritmo que mais realiza comparações. Neste valores de n o Quick Sort se torna o mais eficiente, seguido pelo Heap Sort. Assim, observamos um comportamento de inversão na colocação dos algoritmos quanto ao número médio de comparações em vetores aleatórios para os diferentes valores de n testados.

5.2.2 Análise em Relação às Quantidades de Movimentações de Registros

OBS: O Merge Sort tem um número fixo de movimentações em função do valor de n, independentemente do estado inicial do vetor. Apesar disso, iremos compará-lo normalmente com os demais algoritmos logarítmicos abaixo.

O Quick Sort é o algoritmo que menos realiza movimentações para qualquer valor de n e qualquer estado inicial do vetor, e é, portanto, o algoritmo mais eficiente neste quesito. Classificaremos os demais métodos, excetuando este, abaixo:

- **Vetores Ordenados**
  Para vetores ordenados, qualquer que seja n, o Shell Sort é o algoritmo mais eficiente em movimentações, seguido pelo Heap e por último pelo Merge Sort. O número de movimentações sempre equivale ao número de comparações feitas.
- **Vetores Inversamente Ordenados**
  Com esta configuração inicial de vetor o melhor método é o Heap Sort, que vence o Shell Sort e o Merge, nessa ordem.

Observamos, até o momento, que para ambos estes estados originais de vetor o Merge Sort é o que apresenta pior desempenho, que o Shell Sort é o melhor para vetores ordenados e que o Heap vence para vetores inversamente ordenados. Vejamos como o Merge ascende em relação ao Shell Sort para vetores aleatórios:

- **Vetores Aleatórios**
  Para este estado original de vetor o Heap Sort é o que apresenta menor número médio de movimentações de registros. O Merge Sort passa a ser o segundo melhor, distando grandemente do último e pior algoritmo para este cenário, o Shell Sort. O Shell Sort é drasticamente pior que os demais neste caso, sendo, em média, cerca de 2x pior do que o Merge e 7x pior do que o Quick Sort em número médio de movimentações de registros.

5.2.3 Análise em Relação aos Tempos de Execução

Independentemente do estado original do vetor, para $n = 100$ e $n = 1000$, não há diferença perceptível em tempo de execução entres os algoritmos.
Para $n = 10000$ temos a seguinte ordem, do mais rápido, em média, ao mais lento: Heap Sort, Quick Sort,

Shell Sort e Merge Sort. A diferença, porém, entre os tempos médio são de milésimos de segundo ou até mesmo de 0,1 milésimos de segundo, uma diferença praticamente desprezível.
Para $n = 100000$, o Quick Sort foi o mais veloz para todos os estado originais do vetor. Entre os demais algoritmos temois: para vetores ordenados, o Shell foi o mais rápido, assim como para inversamente ordenados, porém, para o caso aleatório, o tempo de execução médio do Merge foi o menor. Assim, o mais lento em tempo de execução médio foi o Heap Sort.

5.3 RadixSort

O Radix Sort é classificado à parte pois este algoritmo não usa comparações diretas de chaves para ordenar o vetor, e sim uma lógica de ordenação que utiliza de filas para ordenar os elementos do bit/algarismo menos significativo para o mais. Comparemos-o, então, aos demais métodos de ordenação:
OBS: Apesar de não usar comparações diretas de chaves, o contador de comparações é incrementado uma vez que dentro do algoritmo há um laço em que busca o maior elemento dentro do vetor. Isso ocorre dentro do Radix mas não caracteriza a comparação entre dois elementos distintos do vetor.

5.3.1 Análise em Relação às Quantidades de Comparações

O Radix Sort sempre realiza, independentemente da configuração inicial do array, no processo descrito acimo, $n - 1$ comparações, equivalente ao Bubble Sort aprimorado e ao Insertion Sort para vetores ordenados. Logo, ele sempre é o método que realiza menos comparações de chaves, sendo extremamente eficiente neste quesito, uma vez que este processo tem custo $O(n)$.

5.3.2 Análise em Relação às Quantidades de Movimentações de Registros

O número de movimentações de registros depende da quantidade de dígitos k do maior número segundo a expressão:

$$Quantidade\ de\ Movimentacoes\ de\ registros = 2 \cdot n \cdot k$$

Desta maneira, para um entrada do tamanho $n = 100000$, sabendo que o maior número possível é 700000 (gerador de aleatórios gera números de 1 a $7 \cdot n$), o número máximo possível de movimentações é de $2 \cdot n \cdot 6 = 12 \cdot 100000 = 1200000$, já que 700000 possui $k = 6$ dígitos.
Assim, o Radix apresenta números intermediários de movimentações, sendo sempre inferior à Contagem de Menores (apenas $2 \cdot n$ movimentações), ao Quick Sort e ao Selection Sort.

5.3.3 Análise em Relação aos Tempos de Execução

Quanto ao tempo de execução, o Radix é extremamente eficiente, mantendo números baixos até para n grande. Isso em partes se deve ao fato de que o número máximo possível de dígitos é de 6, impedindo que a complexidade do algoritmo exceda $O(6 \cdot n)$. Assim, seu tempo de execução se assemelha ao Quick Sort, sendo um pouco menos veloz, no geral.

---

7. Conclusões

A partir dos dados obtidos e das análises feitas acima, concluímos:

- **Melhor Desempenho (O Algoritmo Mais Eficiente)**

O algoritmo que demonstrou o **melhor desempenho geral** para grandes volumes de dados ($n = 100000$), considerando a velocidade (Tempo de Execução), foi o **Quick Sort**. O Quick Sort se destacou como o mais veloz em tempo de execução para $n = 100000$ em todos os estados originais do vetor (Ordenado, Inversamente Ordenado e Aleatório). Além disso, ele é o que menos realiza movimentações de registros entre todos os métodos logarítmicos e intermediários.
No entanto, o **Radix Sort** ($O(k \cdot n)$) merece menção como o mais eficiente em um aspecto específico:
Melhor Desempenho na Métrica de Comparações: O Radix Sort é o algoritmo que realiza o menor

número de comparações de chaves, com complexidade $O(n)$, realizando apenas $n-1$ comparações no processo de busca do maior elemento.

Vale destacar que o melhor caso absoluto é atingido pelo Bubble Sort aprimorado quando o vetor está ordenado, executando com complexidade $O(n)$, devido ao término precoce após uma iteração.

- **Pior Desempenho (O Algoritmo Mais Ineficiente)**

O algoritmo com o **pior desempenho absoluto** foi o **Bubble Sort**, especificamente quando confrontado com sua pior configuração de entrada: o Vetor Inversamente Ordenado.
Para $n = 100000$ com um vetor inversamente ordenado, o Bubble Sort registrou um tempo de execução de $27,733$ segundos, o valor mais alto observado em toda a análise.
O **Rank Sort (Contagem de Menores)** também se destaca por sua **ineficiência consistente** no quesito temporal dentro do grupo quadrático ($O(n^2)$), sendo o pior para vetores Ordenados e Aleatórios, e o segundo pior para vetores Inversamente Ordenados. Este método é extremamente ineficiente e cerca de 3 vezes mais lento que o segundo algoritmo mais lento (Selection Sort) para vetores ordenados.

Por que o Bubble Sort é o Pior?

O Bubble Sort atinge seu pior desempenho na situação de vetor inversamente ordenado devido à sua **complexidade temporal quadrática** $O(n^2)$ combinada com a necessidade de realizar o número máximo de comparações e trocas.

1. **Complexidade $O(n^2)$:** O tempo de execução cresce exponencialmente com o quadrado do tamanho da entrada. Para $n = 100000$, n^2 é 10 bilhões, o que torna o algoritmo inviável.
2. **Máximo de Movimentações:** No caso do vetor inversamente ordenado, **cada par** de elementos que precisa ser ordenado está na posição errada. Isso força o algoritmo a realizar o **máximo teórico de comparações e trocas** (aproximadamente $n^2/2$ comparações e $n^2/2$ trocas), atingindo cerca de $4,9$ bilhões de movimentações de registros para $n = 100000$. É essa quantidade massiva de operações que resulta no tempo de execução de quase meio minuto.